

РЕФЕРАТ

РПЗ содержит 64 с., 18 рис., 29 табл., 7 приложений, 23 источника.

Ключевые слова: база данных, спортивная аналитика, NBA, реляционная модель, PostgreSQL, Redis, кэширование, метрики эффективности, PER, BPM, REST API, ролевая модель, индексация, производительность.

Объект исследования — процессы накопления, обработки и анализа массивов спортивной статистики NBA.

Предмет исследования — структура реляционной базы данных и алгоритмы вычисления продвинутых метрик эффективности баскетболистов.

Цель работы — разработка базы данных для хранения и анализа статистики игроков и команд Национальной баскетбольной ассоциации с реализацией автоматизированных расчётов продвинутых метрик эффективности.

В аналитическом разделе проведён сравнительный анализ существующих систем спортивной статистики, сформулированы функциональные и нефункциональные требования, формализованы правила расчёта пяти продвинутых метрик (TS%, eFG%, USG%, PER, BPM), выделены восемь сущностей предметной области, построены ER-диаграмма и диаграмма вариантов использования, обоснован выбор реляционной модели данных со строковым хранением.

В конструкторском разделе спроектирована реляционная схема из восьми таблиц, трёх представлений и шести индексов, доказано соответствие 3НФ, разработаны алгоритмы расчёта метрик и ограничений целостности (блок-схемы приведены в приложении А), определена ролевая модель из трёх ролей, спроектировано REST API из 20 маршрутов и механизм кэширования по паттерну Cache-Aside.

В технологическом разделе обоснован выбор стека (PostgreSQL 15, Redis 7, FastAPI, React 18), реализована физическая структура базы данных, разработаны хранимые PL/pgSQL-функции и триггеры (приложение Г), приведены примеры SQL-запросов (приложение В), база данных наполнена реальной статистикой NBA объёмом более 150 000 фактов, реализован пользовательский интерфейс (приложение Е), проведено регрессионное тестирование с верификацией по классам эквивалентности.

В исследовательском разделе измерено влияние стратегий индексации, предвычисленной витрины и кэширования Redis на производительность аналитических запросов; проведено нагрузочное тестирование при конкурентном доступе до 100 одновременных подключений; сформулированы рекомендации по оптимизации.

Разработанный программный комплекс применим в качестве аналитической платформы для профильных специалистов спортивных организаций.

Оглавление

РЕФЕРАТ	4
СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	10
1 Аналитический раздел	11
1.1 Анализ предметной области	11
1.2 Сравнительный анализ существующих систем	11
1.3 Требования к проектируемой системе	12
1.3.1 Функциональные требования	12
1.3.2 Нефункциональные требования	13
1.3.3 Ограничения	13
1.4 Метрики эффективности и правила их расчёта	13
1.5 Сущности предметной области, роли пользователей и варианты использования	15
1.5.1 Сущности и связи	15
1.5.2 Роли пользователей и варианты использования	16
1.6 Выбор модели данных и архитектуры хранилища	17
1.6.1 Выбор логической модели данных	17
1.6.2 Выбор физической архитектуры	18
2 Конструкторский раздел	20
2.1 Логическое проектирование базы данных	20
2.1.1 Соответствие нормальным формам	24
2.1.2 Проектирование представлений	24
2.1.3 Проектирование индексов	25
2.2 Проектирование ограничений целостности	25
2.3 Проектирование алгоритмов обработки данных	26
2.3.1 Алгоритмы расчёта метрик игроков	26
2.3.2 Алгоритм агрегирования командной статистики	27
2.3.3 Алгоритм массового обновления агрегатов	27
2.4 Проектирование ролевой модели доступа	28
2.5 Проектирование программного обеспечения и кэширования	28

2.5.1	Архитектура приложения	28
2.5.2	Спецификация REST API	29
2.5.3	Кэширование результатов запросов	31
3	Технологический раздел	32
3.1	Выбор и обоснование средств реализации	32
3.1.1	СУБД	32
3.1.2	Кэширование	32
3.1.3	Серверный слой	32
3.1.4	Клиентский слой	33
3.2	Физическая реализация базы данных	33
3.2.1	Создание таблиц и ограничений	33
3.2.2	Реализация индексов	33
3.2.3	Загрузка тестовых данных	34
3.2.4	Примеры запросов к базе данных	34
3.3	Реализация логики предметной области в СУБД	37
3.3.1	Хранимые функции расчёта метрик	37
3.3.2	Триггеры логической целостности	37
3.4	Реализация ролевой модели доступа	37
3.5	Реализация программного обеспечения и интерфейса	37
3.5.1	Серверное приложение	37
3.5.2	Кэширование	38
3.5.3	Пользовательский интерфейс	39
3.6	Тестирование и верификация аналитических данных	39
4	Исследовательский раздел	43
4.1	Цель и методика исследования	43
4.2	Влияние объёма данных и стратегий индексации	43
4.2.1	Влияние объёма данных на время выполнения запроса	43
4.2.2	Исследование стратегий индексации	43
4.3	Сравнение витрины и прямой агрегации по таблице фактов	44
4.4	Исследование эффективности кэширования	45
4.5	Исследование производительности при параллельной нагрузке	46
4.6	Анализ результатов и рекомендации	46
	ЗАКЛЮЧЕНИЕ	48
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	49
	ПРИЛОЖЕНИЯ	51
	Приложение А. Блок-схемы алгоритмов	51

Приложение Б. Скрипты создания структуры базы данных	56
Приложение В. Примеры SQL-запросов	57
Приложение Г. Скрипты создания функций и процедур	59
Приложение Д. Исходный код механизма кэширования	61
Приложение Е. Пользовательский интерфейс программного комплекса	62
Приложение Ж. Презентация	64

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Спортивные метрики и обозначения

AST	передачи (assists)
BLK	блок-шоты (blocks)
BPM	вклад игрока (Box Plus/Minus)
eFG%	эффективный процент с игры (effective field goal percentage)
FG3A	попытки трёхочкового броска
FG3M	попадания трёхочковых бросков
FGA	попытки броска с игры (field goals attempted)
FGM	попадания с игры (field goals made)
FTA	попытки штрафного броска (free throws attempted)
FTM	попадания штрафных бросков (free throws made)
MP	минуты на площадке (minutes played)
NBA	Национальная баскетбольная ассоциация
PER	рейтинг эффективности игрока (Player Efficiency Rating)
PF	персональные фолы (personal fouls)
PTS	очки (points)
REB_DEF	подборы в защите
REB_OFF	подборы в нападении
STL	перехваты (steals)
TmFGA	командные попытки броска с игры
TmFTA	командные попытки штрафного броска
TmMP	суммарные минуты команды
TmTOV	командные потери
TOV	потери (turnovers)
TRB	подборы суммарные (total rebounds)
TS%	истинный процент бросков (true shooting percentage)
USG%	доля владений в атаке (usage percentage)

Технические

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
B-tree	сбалансированное дерево (balanced tree)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DDL	Data Definition Language
ER	Entity-Relationship
ETL	Extract, Transform, Load
FK	внешний ключ (Foreign Key)
HTAP	Hybrid Transactional/Analytical Processing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KPI	ключевые показатели эффективности (Key Performance Indicators)
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
P95	95-й перцентиль
PK	первичный ключ (Primary Key)
PL/pgSQL	процедурный язык PostgreSQL
REST	Representational State Transfer
SPA	одностраничное приложение (Single Page Application)
SQL	Structured Query Language
СУБД	система управления базами данных
UML	Unified Modeling Language
URL	Uniform Resource Locator
UPSERT	операция вставки с обновлением (UPDATE + INSERT)
1НФ / 2НФ / 3НФ	первая / вторая / третья нормальная форма

ВВЕДЕНИЕ

Современный профессиональный спорт тесно связан с анализом статистики. Для оценки игроков уже недостаточно базовых показателей — используются сложные метрики, которые рассчитываются на основе большого объема данных о матчах и игроках. Обработка такой информации требует высоких вычислительных ресурсов и надежной системы хранения данных. Кроме того, важно обеспечить корректность вычислений, защиту данных и быстрое выполнение аналитических запросов. Поэтому возникает необходимость в проектировании специализированной реляционной базы данных, оптимизированной для задач спортивной аналитики.

Цель работы: разработка базы данных для хранения и анализа статистики игроков и команд Национальной баскетбольной ассоциации с реализацией автоматизированных расчётов продвинутых метрик эффективности.

Для достижения поставленной цели сформулированы следующие задачи:

- 1) провести системный анализ предметной области, выявить ограничения существующих решений и формализовать математические правила вычисления метрик;
- 2) спроектировать логическую и физическую структуру реляционной базы данных, алгоритмы процедурной обработки фактов и ролевую модель доступа;
- 3) спроектировать программную архитектуру серверного слоя и механизм кэширования для обеспечения высокой пропускной способности;
- 4) реализовать спроектированную систему, выполнить загрузку исторических данных и верифицировать корректность вычисляемых показателей;
- 5) провести нагрузочное тестирование и исследовать влияние индексов, материализованных витрин и кэширования на производительность аналитических выборок.

Объект исследования: процессы накопления, обработки и анализа массивов спортивной статистики.

Предмет исследования: структура реляционной базы данных и алгоритмы вычисления продвинутых метрик эффективности баскетболистов.

1 Аналитический раздел

1.1 Анализ предметной области

Официальная статистика Национальной баскетбольной ассоциации предоставляет открытые структурированные данные по каждому матчу [1]. Для каждого матча фиксируются итоговый счёт, составы команд и числовые показатели по каждому игроку: очки, подборы, передачи, блок-шоты, перехваты и иные. На основе первичных числовых показателей вычисляются рейтинги, сопоставляются сезоны и выявляются тенденции развития игры.

Для оценки результативности игроков используются продвинутое метрики: истинный процент бросков, эффективный процент с игры, доля владений в атаке, рейтинг эффективности и вклад игрока [2]. Их расчёт требует хранения непротиворечивых первичных данных по матчам, агрегирования по сезонам и применения фиксированных правил вычисления.

Обработка спортивной статистики требует строгого контроля над целостностью данных: сумма очков игроков должна совпадать с командным счётом, агрегаты по сезонам должны вычисляться из одних и тех же первичных фактов, а параллельный доступ не должен нарушать консистентность записей. Это определяет необходимость разработки специализированного хранилища данных с поддержкой транзакционной целостности и декларативной валидации бизнес-правил.

1.2 Сравнительный анализ существующих систем

Проведён анализ функциональных возможностей публичных систем статистики NBA: NBA Stats [1], Basketball-Reference [3] и ESPN [4].

Проектируемая система предназначена для внутреннего использования профильными специалистами. Функционала базового просмотра агрегированных данных на публичных ресурсах для решения глубоких аналитических задач недостаточно. Исходя из требований к проектируемой платформе, сформированы следующие критерии сравнения существующих решений:

- прозрачность расчёта метрик — наличие открытых математических формул критически важно для верификации вычислений, тогда как коммерческие системы зачастую скрывают применяемую методологию [2];
- доступ к первичным данным через API — возможность программного извлечения фактов требуется для построения произвольных аналитических выборок, не ограниченных стандартным пользовательским интерфейсом;
- локальное хранение и воспроизводимость — требование гарантирует неизменность исторической статистики и полную независимость вычислений от доступности внешних сетевых ресурсов;
- ролевое разграничение доступа — механизм разделения привилегий необходим для защиты внутренней аналитики и исходных фактов от несанкционированной модифика-

ции [5].

В таблице 1.1 представлен сравнительный анализ существующих систем по выделенным критериям.

Таблица 1.1 — Сравнительный анализ существующих систем по функциональным возможностям

Критерий	NBA.com Stats	Basketball-Reference	ESPN Stats
Просмотр статистики матчей	+	+	+
Фильтрация по сезону, позиции, команде	частично	+	частично
Расчёт TS%, eFG%, PER, BPM	+(формулы скрыты)	+(формулы открыты [6, 7])	частично
Доступ к сырым данным через API (произвольные запросы)	–	–	–
Интерактивное сравнение стат. профилей игроков	частично	–	+
Ролевое разграничение доступа	–	–	–
Локальное хранение, воспроизводимость расчётов	–	–	–

Анализ показывает, что NBA.com Stats и ESPN предоставляют данные через готовый пользовательский интерфейс с непрозрачными формулами расчёта метрик. Basketball-Reference публикует описание своих формул [6, 7], однако не предоставляет открытого API для произвольных аналитических запросов. Ни одна из систем не поддерживает ролевого разграничения доступа и воспроизводимого локального хранения данных. Выявленные ограничения определяют требования к проектируемой системе, сформулированные в следующем разделе.

1.3 Требования к проектируемой системе

1.3.1 Функциональные требования

- 1) хранение справочных данных: игровых позиций, сезонов, клубов NBA;
- 2) хранение профилей игроков: персональные и антропометрические данные, позиция, сведения о драфте;
- 3) хранение данных о матчах: сезон, команды-участники, дата, итоговый счёт, статус;
- 4) хранение детализированной статистики игрока в матче: очки, подборы, передачи, броски с игры и штрафные, потери, фолы, время на площадке;
- 5) хранение агрегированной статистики игрока за сезон: средние значения, процентные показатели, расчётные метрики;
- 6) хранение истории выступлений игрока за клубы по сезонам;
- 7) расчёт командной статистики в рамках матча и сезона на основе индивидуальных показателей игроков;

- 8) автоматизированный расчёт метрик эффективности: истинный процент бросков, эффективный процент с игры, доля владений в атаке, рейтинг эффективности и вклад игрока на основе первичных фактов матчей;
- 9) предоставление сводных рейтингов игроков и командной статистики.

1.3.2 Нефункциональные требования

- 1) производительность: время отклика на запросы чтения не должно превышать порог мгновенного восприятия в 100 мс [8];
- 2) объём данных: система должна выдерживать нагрузку не менее 100 000 записей в таблице фактов;
- 3) целостность: база данных должна соответствовать требованиям ACID, обеспечивая ссылочную и логическую целостность на уровне СУБД;
- 4) безопасность: ролевое разграничение доступа (администратор, аналитик, читатель), реализованное средствами СУБД на уровне таблиц и представлений;
- 5) масштабируемость чтения: время отклика на повторяющиеся аналитические запросы при конкурентном доступе не должно превышать 100 мс при нагрузке до 50 одновременных подключений, что соответствует оценке совокупной аудитории системы: аналитический отдел, тренерский штаб и скауты клуба NBA.

1.3.3 Ограничения

- 1) глубина хранения данных ограничена пятью завершёнными регулярными сезонами с 2019 по 2024 г.;
- 2) пользовательский интерфейс предназначен для просмотра и анализа данных, а не для ввода первичной статистики в режиме реального времени.

В рамках опытной эксплуатации и нагрузочного тестирования историческая выборка ограничена пятью завершёнными регулярными сезонами с 2019 по 2024 г., что обеспечивает объём выборки более 150 000 фактов и репрезентативность метрик.

1.4 Метрики эффективности и правила их расчёта

Выбор конкретных метрик (TS%, eFG%, USG%, PER, BPM) обусловлен необходимостью перехода от базовой статистики (очки, подборы) к комплексной оценке эффективности игрока. Согласно современной методологии спортивной аналитики [2, 10], абсолютные показатели искажаются темпом игры и количеством сыгранного времени. Использование TS% и eFG% решает проблему математической неравноценности штрафных, двухочковых и трёхочковых бросков. Метрика USG% необходима для нормализации доли задействования игрока в атаке. Интегральные показатели PER [11] и BPM [7] позволяют свести многомерный профиль баскетболиста к единому числовому значению, что является стандартом де-факто для задач ранжирования

игроков в системах спортивной аналитики.

Командная статистика не выделяется в отдельную сущность: она является производной от индивидуальных показателей и вычисляется динамически суммированием по всем игрокам команды в конкретном матче.

Эффективный процент с игры. Метрика корректирует ценность трёхочкового броска. Согласно методологии NBA [9], формула имеет вид:

$$eFG\% = \frac{FGM + 0,5 \cdot FG3M}{FGA}. \quad (1.1)$$

где

FGM — попадания с игры;

FG3M — попадания трёхочковых бросков;

FGA — попытки броска с игры.

Истинный процент бросков. Показатель объединяет броски с игры и штрафные броски в единую метрику эффективности. Множитель 0,44 в знаменателе является эмпирически выведенным коэффициентом Дина Оливера, учитывающим долю штрафных бросков, завершающих владение [10]:

$$TS\% = \frac{PTS}{2 \cdot (FGA + 0,44 \cdot FTA)}, \quad (1.2)$$

где

PTS — очки;

FGA — попытки броска с игры;

FTA — попытки штрафного броска.

Доля владений в атаке. Оценивает процент командных владений, которые завершает конкретный игрок, находясь на площадке [2]. В базе данных показатель хранится как доля [0; 1]; в интерфейсе отображается в процентах ($\times 100$). Формула:

$$USG = \frac{(FGA + 0,44 \cdot FTA + TOV) \cdot (TmMP/5)}{MP \cdot (TmFGA + 0,44 \cdot TmFTA + TmTOV)}, \quad (1.3)$$

где

FGA, FTA, TOV — показатели игрока;

TmFGA, TmFTA, TmTOV, TmMP — командные агрегаты;

MP — минуты игрока на площадке.

Деление TmMP на 5 нормирует командное время с учётом того, что на площадке одновременно находятся пять игроков.

Рейтинг эффективности игрока. Оригинальная формула разработана Джоном Холлинджером [11]. В настоящей работе применяется её упрощённая версия без позиционных поправок, методология которой описана на портале Basketball-Reference [6]. Подборы TRB = REB_OFF + REB_DEF. Ненормализованный вклад приводится к шкале «за 48 минут», затем

масштабируется к среднему по лиге, равному 15:

$$uPER_i = (PTS + TRB + AST + STL + BLK - TOV - PF - (FGA - FGM) - 0,44 \cdot (FTA - FTM)) \cdot (48/MP_i), \quad (1.4)$$

$$lg_aPER = AVG(uPER_i) \quad \text{по игрокам с } MP_i > 100, \quad (1.5)$$

$$PER_i = uPER_i \cdot \frac{15}{lg_aPER}. \quad (1.6)$$

Минимальный порог $MP_i \geq 50$ минут за сезон; для расчёта lg_aPER учитываются игроки с $MP_i > 100$ минут.

Вклад игрока. Метрика разработана Дэниелом Майерсом [7]. В системе применяется упрощённая линейная аппроксимация BPM с нормировкой показателей на 100 минут ($X_{100} = X/MP \cdot 100$) и учётом перехватов, блоков и фолов. Итоговая формула:

$$raw_i = 0,123 \cdot PTS_{100} + 0,234 \cdot TRB_{100} + 0,689 \cdot AST_{100} + 0,445 \cdot STL_{100} + 0,407 \cdot BLK_{100} - 0,605 \cdot TOV_{100} - 0,086 \cdot PF_{100}, \quad (1.7)$$

$$BPM_i = raw_i - AVG(raw_i) \quad \text{по игрокам с } MP_i > 100. \quad (1.8)$$

Среднее BPM по лиге после вычитания среднего равно 0. Минимальный порог для расчёта показателя игрока — $MP_i \geq 50$ минут за сезон.

1.5 Сущности предметной области, роли пользователей и варианты использования

1.5.1 Сущности и связи

Для расчёта метрик, описанных в разделе 1.4, необходимо хранить атомарные показатели игрока в каждом матче. Исходя из сформулированных в п. 1.3 функциональных требований, границы предметной области ограничены показателями игроков и команд. На основе этого выделены следующие сущности предметной области:

- 1) позиция — справочник игровых позиций в номенклатуре NBA;
- 2) сезон — справочная сущность, определяющая временные рамки игрового года и тип турнира: регулярный чемпионат, плей-офф или предсезонные матчи;
- 3) команда — клуб NBA: конференция, арена;
- 4) игрок — персональные и антропометрические данные, позиция, сведения о драфте;
- 5) матч — сезон, команды-участники, дата, счёт, статус;
- 6) статистика игрока в матче — числовые показатели конкретного игрока в конкретной игре; основная таблица фактов предметной области;
- 7) статистика игрока за сезон — агрегаты и расчётные метрики по тройке «игрок –

сезон – команда»;

8) история выступлений — связь «игрок – команда – сезон» с датами и типом контракта.

Связь «многие-ко-многим» между игроками и матчами реализуется через сущность «Статистика игрока в матче»: один игрок участвует во многих матчах, в одном матче фиксируется статистика многих игроков [5, 12].

Связь матча с сезоном и командами на ER-диаграмме представлена тремя бинарными связями: «Проходит в» связывает Сезон (1) и Матч (М); «Дом. команда» и «Гост. команда» связывают Команду (1) с Матчем (М) для хозяев и гостей соответственно [12].

Концептуальная ER-диаграмма в нотации Чена приведена на рисунке 1.1; на ней отражены сущности предметной области и бинарные связи между ними.

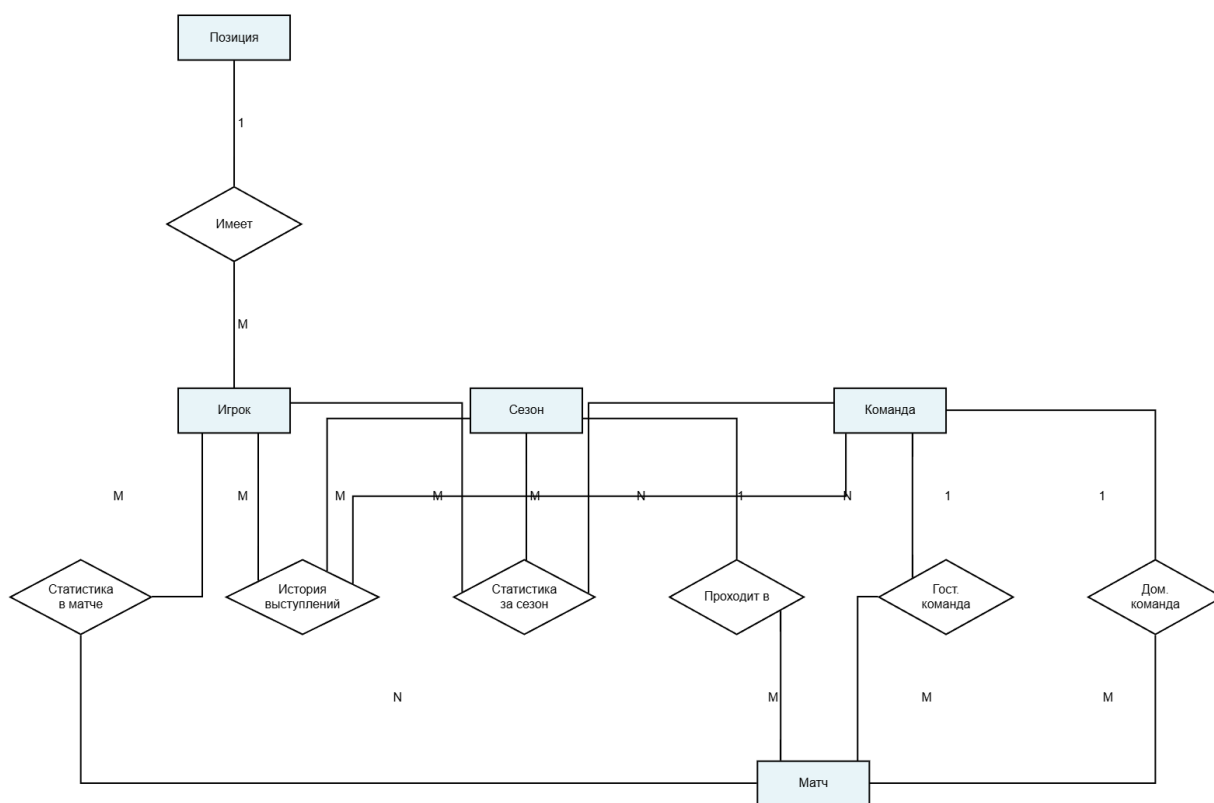


Рисунок 1.1 — Концептуальная ER-диаграмма

1.5.2 Роли пользователей и варианты использования

Система предусматривает три роли пользователей.

- 1) администратор — полное управление структурой и содержимым хранилища: создание и изменение схемы, загрузка и сопровождение данных, резервное копирование;
- 2) аналитик — чтение данных и запуск расчётных процедур для получения производных показателей;
- 3) читатель — доступ к агрегированным данным через представления; изменение содержимого базовых таблиц недоступно.

Соответствие ролей и допустимых операций задаётся тремя вариантами использования.

- 1) просмотр статистики — получение сведений о матчах, игроках и командах, агрегированных показателях и рейтингах; доступно всем ролям;
- 2) расчёт метрик — вычисление продвинутых показателей на основе первичных данных; доступно аналитику и администратору;
- 3) управление данными — загрузка данных о матчах, обновление справочников, изменение схемы; доступно только администратору.

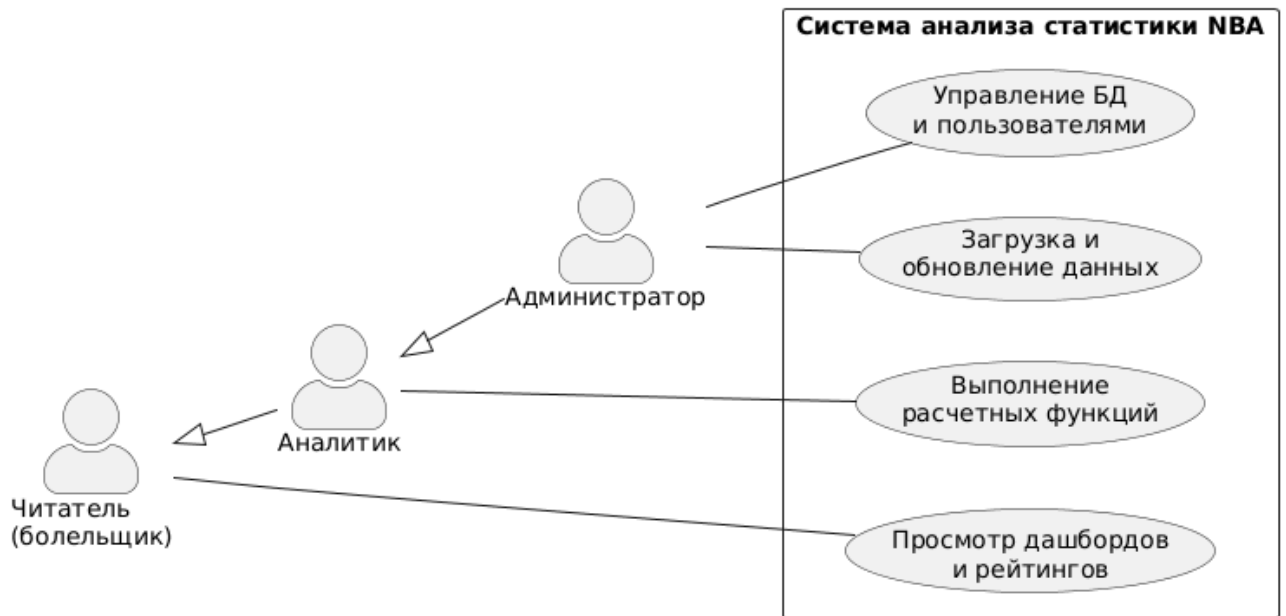


Рисунок 1.2 — Диаграмма вариантов использования

1.6 Выбор модели данных и архитектуры хранилища

Задача разделена на выбор логической модели данных и определение физической архитектуры хранилища [13].

1.6.1 Выбор логической модели данных

Графовая модель данных. Представляет информацию в виде узлов и связей. Данная модель оптимальна для задач обхода связей, однако демонстрирует крайне низкую производительность при доминировании многомерных математических агрегаций по большим массивам фактов. Вычисление средних значений, сумм и процентных показателей по сотням тысяч записей делает графовые СУБД неэффективными для задач спортивной статистики [13].

Документно-ориентированная модель. Хранит данные в виде независимых иерархических структур. Документно-ориентированный подход оптимален для объектов без жёсткой схемы [14]. Первичная статистика матча соответствует иерархической структуре документа, однако агрегация данных по нескольким сезонам и связывание игроков с историей команд требует сложных программных JOIN-операций, которые в документной модели неэффективны.

Реляционная модель данных. Оперирует двумерными таблицами с жёстко заданной схемой [5]. Спортивная статистика характеризуется фиксированной структурой атрибутов. Реляционная алгебра предоставляет аппарат для многомерных агрегаций, а сама модель предотвращает попадание аномальных данных за счёт механизмов ссылочной целостности.

Таблица 1.2 — Сравнительная характеристика логических моделей данных

Критерий	Графовая	Документная	Реляционная
Управление схемой	Отсутствует / Гибкая	Гибкая	Строгая
Математические агрегации фактов	Низкая эффективность	Средняя (через пайплайны)	Высокая (нативная оптимизация)
Связывание сущностей (JOIN)	Обход по ссылкам (быстро)	Затруднено (денормализация)	Нативная поддержка (внешние ключи)
Целостность фактов	На уровне приложения	Поддержка JSON Schema	Строгие констрейнты таблиц

На основе анализа логических моделей выбрана **реляционная модель**, так как данные строго структурированы и требуют сложных агрегаций с объединением справочников.

1.6.2 Выбор физической архитектуры

Выбор физической архитектуры определяется необходимостью балансировки транзакционной и аналитической нагрузок.

Колоночные СУБД. Многократно ускоряют аналитические запросы над миллионами записей за счёт сжатия и векторного выполнения операций [15], однако неэффективны при точечных транзакционных обновлениях связанных сущностей.

Строковые транзакционные СУБД. Ориентированы на гибридную аналитико-транзакционную нагрузку. Гарантируют строгую согласованность данных на уровне физических страниц при регулярном обновлении фактов и справочников [16].

Проектируемая система предполагает гибридный профиль: процедура обновления сезонной статистики выполняет ресурсоёмкие операции слияния для сотен записей, а также требует ретроспективной корректировки протоколов матчей. При целевом объёме таблицы фактов порядка 200 000 строк (5 сезонов) классическая строковая реляционная СУБД способна выполнять аналитические запросы без существенных задержек с помощью B-tree индексов. Отставание в скорости чтения сводных таблиц будет полностью нивелировано применением паттерна in-memory кэширования на уровне приложения. Использование колоночной СУБД при масштабах в сотни тысяч строк является архитектурной избыточностью.

Таким образом, для реализации хранилища целесообразно использовать реляционную СУБД со строковой физической архитектурой, дополненную in-memory кэшированием.

Вывод

Сформулированы функциональные и нефункциональные требования и ограничения; определён перечень метрик и правила их вычисления. Выделены восемь сущностей, установлены связи «многие-ко-многим» через таблицы фактов, описаны три пользовательские роли.

По результатам сравнения логических и физических моделей данных для проектируемой системы выбрана реляционная модель со строковым хранением. В отличие от документоориентированных и графовых моделей, она позволяет эффективно выполнять многомерные агрегации фактов, а в отличие от колоночных систем — обеспечивает полную транзакционную целостность (ACID) при точечных и множественных обновлениях связанных таблиц.

Сравнительный анализ существующих систем (NBA.com Stats, Basketball-Reference, ESPN Stats) подтвердил отсутствие у них произвольных аналитических запросов, управляемого ролевого доступа и воспроизводимого локального хранения данных, что обуславливает актуальность и целесообразность разработки специализированной базы данных. ER-диаграмма и диаграмма вариантов использования приведены на рисунках 1.1 и 1.2.

2 Конструкторский раздел

2.1 Логическое проектирование базы данных

Реляционная схема (рисунок 2.1) включает восемь таблиц. Факты хранятся в game_player_stats, денормализованная витрина player_season_stats ускоряет аналитические выборки. Справочники используют суррогатные ключи.

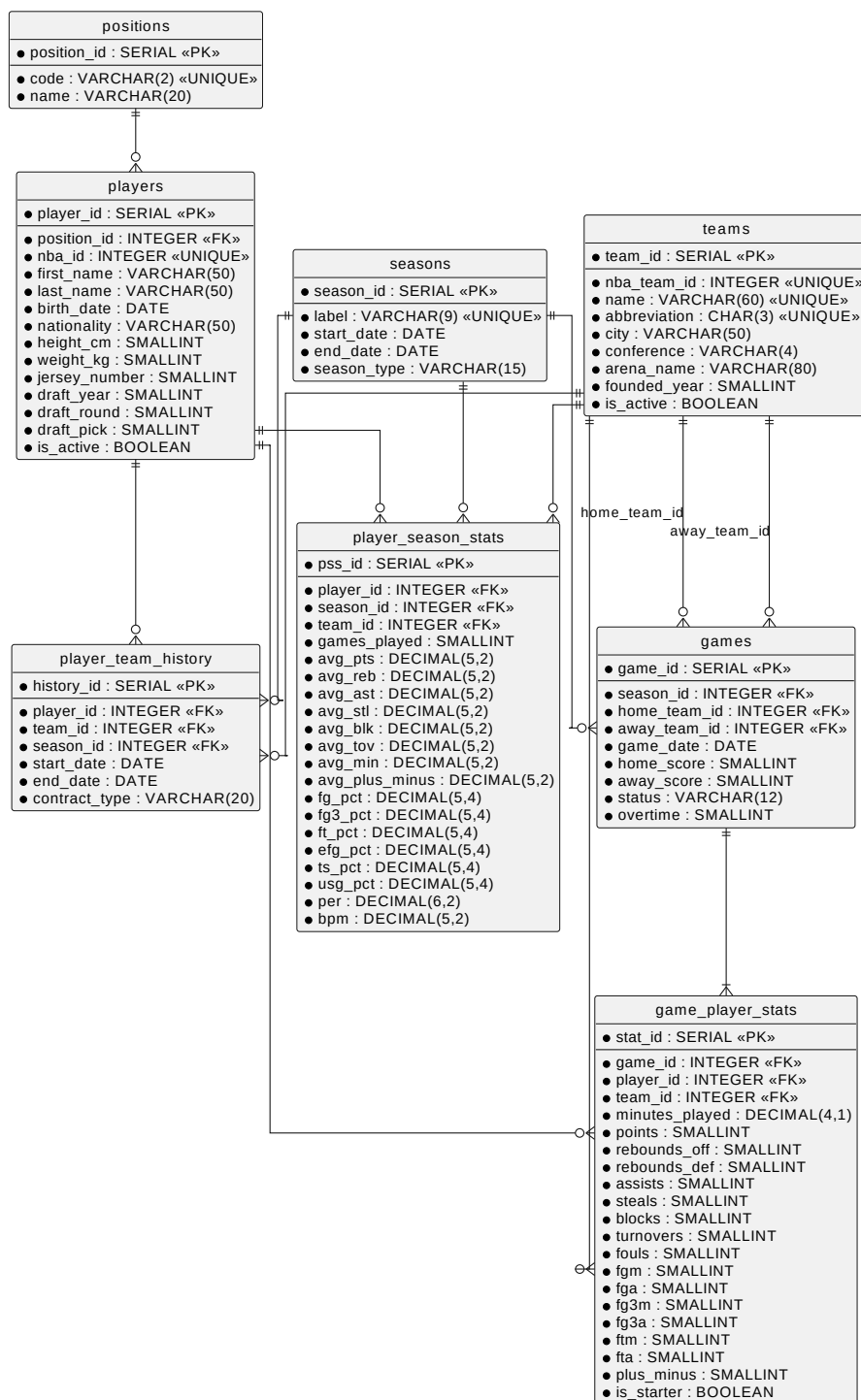


Рисунок 2.1 — Реляционная схема базы данных

Для нормализованного хранения игровых позиций спроектирована таблица `positions` (табл. 2.1).

Таблица 2.1 — Справочник игровых позиций

Атрибут	Тип	Ключ / назначение
<code>position_id</code>	SERIAL	PK
<code>code</code>	VARCHAR(2)	UNIQUE; код PG, SG, SF, PF, C
<code>name</code>	VARCHAR(20)	полное название позиции

Интервалы проведения соревнований описываются таблицей `seasons` (табл. 2.2).

Таблица 2.2 — Справочник сезонов

Атрибут	Тип	Ключ / назначение
<code>season_id</code>	SERIAL	PK
<code>label</code>	VARCHAR(9)	UNIQUE; метка сезона, в частности формата 2023-24
<code>start_date</code>	DATE	дата начала сезона
<code>end_date</code>	DATE	дата окончания сезона
<code>season_type</code>	VARCHAR(15)	Regular, Playoff, Preseason

Информация о баскетбольных клубах и их домашних аренах хранится в таблице `teams` (табл. 2.3).

Таблица 2.3 — Справочник команд

Атрибут	Тип	Ключ / назначение
<code>team_id</code>	SERIAL	PK
<code>nba_team_id</code>	INTEGER	UNIQUE; id на NBA.com
<code>name</code>	VARCHAR(60)	UNIQUE; полное название
<code>abbreviation</code>	CHAR(3)	UNIQUE; аббревиатура
<code>city</code>	VARCHAR(50)	город базирования
<code>conference</code>	VARCHAR(4)	CHECK IN ('East', 'West')
<code>arena_name</code>	VARCHAR(80)	домашняя арена
<code>founded_year</code>	SMALLINT	год основания
<code>is_active</code>	BOOLEAN	признак активной команды

Для хранения персональных и антропометрических данных баскетболистов разработана таблица `players` (табл. 2.4).

Таблица 2.4 — Справочник игроков

Атрибут	Тип	Ключ / назначение
player_id	SERIAL	PK
nba_id	INTEGER	UNIQUE; id на NBA.com
first_name	VARCHAR(50)	имя
last_name	VARCHAR(50)	фамилия
birth_date	DATE	дата рождения
nationality	VARCHAR(50)	гражданство
height_cm	SMALLINT	рост (см)
weight_kg	SMALLINT	вес (кг)
position_id	INTEGER	FK → positions
jersey_number	SMALLINT	номер на майке (0–99)
is_active	BOOLEAN	действующий игрок
draft_year	SMALLINT	год драфта
draft_round	SMALLINT	раунд драфта (1 или 2)
draft_pick	SMALLINT	номер выбора на драфте

Сведения о проведённых и запланированных матчах фиксируются в таблице games (табл. 2.5).

Таблица 2.5 — Матчи

Атрибут	Тип	Ключ / назначение
game_id	SERIAL	PK
season_id	INTEGER	FK → seasons
home_team_id	INTEGER	FK → teams; команда-хозяин
away_team_id	INTEGER	FK → teams; команда-гость
game_date	DATE	дата матча
home_score	SMALLINT	счёт хозяев
away_score	SMALLINT	счёт гостей
status	VARCHAR(12)	Finished, Scheduled, Postponed
overtime	SMALLINT	число овертаймов (0–6)

Атомарные факты результативности игроков в конкретном матче (основная таблица фактов) сохраняются в детализированной таблице game_player_stats (табл. 2.6).

Таблица 2.6 — Факты статистики игрока в матче

Атрибут	Тип	Ключ / назначение
stat_id	SERIAL	PK
game_id	INTEGER	FK → games, ON DELETE CASCADE
player_id	INTEGER	FK → players, ON DELETE RESTRICT
team_id	INTEGER	FK → teams, ON DELETE RESTRICT
minutes_played	DECIMAL(4,1)	минуты на площадке
points	SMALLINT	очки (CHECK ≥ 0)
rebounds_off, rebounds_def	SMALLINT	подборы в нападении и защите
assists, steals, blocks	SMALLINT	передачи, перехваты, блоки
turnovers, fouls	SMALLINT	потери и фолы
fgm, fga, fg3m, fg3a, ftm, fta	SMALLINT	попадания и попытки бросков
plus_minus	SMALLINT	показатель плюс-минус
is_starter	BOOLEAN	признак стартового состава
UNIQUE	—	(game_id, player_id) — гарантирует единственность записи игрока в матче

В таблице game_player_stats пара полей game_id и player_id закреплена ограничением UNIQUE. Это исключает дублирование строк статистики одного игрока в одном матче.

Агрегированные статистические показатели и расчётные метрики эффективности за сезон материализуются в таблице player_season_stats (табл. 2.7).

Таблица 2.7 — Агрегаты статистики за сезон

Атрибут	Тип	Ключ / назначение
pss_id	SERIAL	PK
player_id	INTEGER	FK → players; часть UNIQUE
season_id	INTEGER	FK → seasons; часть UNIQUE
team_id	INTEGER	FK → teams; часть UNIQUE
games_played	SMALLINT	число игр в сезоне
avg_pts, avg_reb, avg_ast	DECIMAL(5,2)	средние за матч (очки, подборы, передачи)
avg_stl, avg_blk, avg_tov, avg_min	DECIMAL(5,2)	перехваты, блоки, потери, минуты
fg_pct, fg3_pct, ft_pct	DECIMAL(5,4)	проценты попаданий (дробь ∈ [0, 1])
avg_plus_minus	DECIMAL(5,2)	средний плюс-минус
efg_pct, ts_pct, usg_pct	DECIMAL(5,4)	eFG%, TS%, USG (доля 0–1; на UI × 100)
per	DECIMAL(6,2)	PER
bpm	DECIMAL(5,2)	BPM
Ограничение UNIQUE: (player_id, season_id, team_id) — не более одной строки агрегата на игрока, сезон и команду		

Трансферы игроков фиксируются в таблице player_team_history (табл. 2.8).

Таблица 2.8 — История выступлений игроков

Атрибут	Тип	Ключ / назначение
history_id	SERIAL	PK
player_id	INTEGER	FK → players; часть UNIQUE
team_id	INTEGER	FK → teams
season_id	INTEGER	FK → seasons
start_date	DATE	дата начала выступления
end_date	DATE	дата окончания (NULL — по сей день)
contract_type	VARCHAR(20)	Standard, Two-Way, 10-Day и др.

2.1.1 Соответствие нормальным формам

Схема приведена к третьей нормальной форме. Ниже выполнена проверка соответствия правилам нормализации [16].

Первая нормальная форма. Требование выполняется: все атрибуты базовых таблиц атомарны и содержат скалярные значения. Сложные структуры данных и массивы отсутствуют. Например, в справочнике игроков полное имя разделено на независимые атрибуты имени и фамилии.

Вторая нормальная форма. Требование выполняется: каждая таблица обладает простым первичным ключом, от которого функционально и полностью зависят все неключевые атрибуты. Частичные зависимости отсутствуют.

Третья нормальная форма. Требование выполняется: транзитивные зависимости устранены путём вынесения справочных данных в независимые отношения. Наличие предварительно вычисленных агрегатов в таблице сезонной статистики обусловлено архитектурным паттерном витрины данных для ускорения аналитических выборок и не нарушает логическую целостность базовых отношений схемы.

2.1.2 Проектирование представлений

Инкапсуляция базовых таблиц и ограничение прямого доступа реализованы через три представления:

- 1) `v_player_rankings` — инкапсулирует операцию JOIN между таблицей агрегатов `player_season_stats` и справочниками `players`, `teams`. Предоставляет денормализованный сводный рейтинг игроков за сезон по базовым и расчётным метрикам, скрывая от клиента сложность объединения таблиц;
- 2) `v_team_stats` — реализует функциональное требование динамического расчёта командной статистики. Выполняет агрегацию данных таблицы `game_player_stats` с применением статистических функций среднего и суммы в разрезе команды и сезона, исключая необходимость хранения избыточной таблицы командных фактов;
- 3) `v_top_players` — обеспечивает безопасность данных для публичной роли `reader`.

Является фильтрованным подмножеством представления `v_player_rankings` с ограничением выборки (`LIMIT`) и сортировкой (`ORDER BY`) лидеров по метрике `PER`.

2.1.3 Проектирование индексов

Производительность аналитических запросов (согласно п. 1.3.2) обеспечена созданием B-tree индексов [17]:

- 1) `idx_gps_player_id` ON `game_player_stats(player_id)` — ускоряет выборку всех выступлений игрока для агрегации сезонной статистики;
- 2) `idx_gps_team_game` ON `game_player_stats(team_id, game_id)` — поддерживает агрегацию командной статистики в представлении `v_team_stats`;
- 3) `idx_games_season_id` ON `games(season_id)` — ускоряет фильтрацию матчей по сезону;
- 4) `idx_games_date` ON `games(game_date)` — обеспечивает диапазонную фильтрацию по дате проведения матча;
- 5) `idx_pss_season` ON `player_season_stats(season_id)` — ускоряет выборку рейтингов за конкретный сезон;
- 6) `idx_players_name` ON `players(last_name, first_name)` — поддерживает поиск игрока по фамилии.

Ограничение `UNIQUE` на `(game_id, player_id)` в таблице `game_player_stats` автоматически создаёт B-tree индекс, покрывающий запросы поиска по паре «матч – игрок». Все индексы реализованы как B-tree — стандартная структура PostgreSQL, оптимальная для равенства и диапазонных предикатов в аналитических запросах.

2.2 Проектирование ограничений целостности

Помимо ограничений первичных и внешних ключей, на уровне СУБД реализована логика защиты бизнес-правил.

Удаление матча вызывает каскадное удаление связанной статистики. Для справочников игроков и команд физическое удаление на уровне внешних ключей запрещено; применяется логическое удаление флагом активности.

ЧЕКС-ограничения. Обеспечивают согласованность бросков: $fg3m \leq fgm$, $fgm \leq fga$; диапазон фолов от 0 до 6. Для таблицы `games` задано ограничение `CHECK (home_team_id <> away_team_id)`. Для `player_team_history` — ограничение `CHECK (start_date <= end_date)`.

Процедурные ограничения целостности (триггеры). На таблице `game_player_stats` реализованы два триггера. Триггер `trg_check_team_score` (`AFTER INSERT OR UPDATE, FOR EACH ROW`) валидирует равенство суммы очков игроков ($\sum points$) командному счёту (`home_score` или `away_score`) для завершённых матчей. Триггер `after_game_player_stats_insert` (`AFTER INSERT, FOR EACH STATEMENT`) вызывает `update_season_stats` для пересчёта витрины. Ограничение по счёту применимо к полностью обработанным данным: игроки, не принявшие

участие в матче, в таблицу фактов `game_player_stats` они не вносятся; загружаемые источники должны обеспечивать атрибуцию всех очков конкретным игрокам.

2.3 Проектирование алгоритмов обработки данных

2.3.1 Алгоритмы расчёта метрик игроков

Расчёт метрик делегирован СУБД. Обработка нулевых знаменателей через возврат неопределённого значения исключает неквалифицированных игроков из агрегаций.

Функция `calculate_ts` рассчитывает истинный процент бросков (формула (1.2)) на основе сезонных агрегатов из `game_player_stats`. При нулевом знаменателе возвращается неопределённое значение. Схема алгоритма вычисления представлена в Приложении А (рисунок А.1).

Функция `calculate_efg` рассчитывает эффективный процент бросков (формула (1.1)) на основе сезонных агрегатов. Отсутствие попыток броска обрабатывается возвратом неопределённого значения для предотвращения деления на ноль. Схема алгоритма вычисления представлена в Приложении А (рисунок А.2).

Функция `calculate_usg` вычисляет долю командных владений, завершаемых игроком, по формуле (1.3). Алгоритм требует двух проходов: суммирования индивидуальных показателей игрока и вычисления командных агрегатов по общим матчам. Функция возвращает `NULL` в трёх случаях: команда игрока не определена, суммарное время на площадке менее одной минуты, либо командный знаменатель равен нулю. Схема алгоритма вычисления представлена в Приложении А (рисунок А.3).

Функция `calculate_per` вычисляет рейтинг эффективности игрока по упрощённой методологии Холлинджера (формулы 1.4–1.6). Алгоритм двухпроходный. На первом проходе вычисляется ненормализованный вклад игрока `uPER` на основе сезонных сумм всех базовых показателей, нормированных на 48 минут. Игроки с суммарным временем менее 50 минут за сезон исключаются из расчёта. На втором проходе вычисляется среднее значение по лиге. При неопределённом лиговом среднем значение принудительно устанавливается в базовый ориентир 15,0. Схема алгоритма вычисления представлена в Приложении А (рисунок А.4).

Функция `calculate_bpm` вычисляет вклад игрока относительно среднего по лиге по упрощённой линейной аппроксимации BPM (формулы 1.7–1.8). Все показатели нормируются на 100 минут игрового времени, после чего взвешиваются фиксированными коэффициентами модели Майерса. Игроки с менее чем 50 минутами дисквалифицируются. Лиговое среднее вычитается из сырого значения, поэтому среднее BPM по лиге после нормировки равно нулю. Схема алгоритма вычисления представлена в Приложении А (рисунок А.5).

Функция-триггер `trg_validate_team_score_row` проверяет бизнес-правило предметной области: сумма очков всех игроков команды в завершённом матче должна совпадать с итоговым командным счётом. Проверка выполняется только для матчей со статусом `Finished`

и заполненным счётом — незавершённые матчи пропускаются без проверки. Для каждой команды, присутствующей в таблице фактов данного матча, вычисляется сумма очков и сравнивается с `home_score` или `away_score`. При несовпадении триггер поднимает исключение и откатывает вставку. Схема алгоритма представлена в Приложении А (рисунок А.6).

Функция-триггер `trigger_update_season_stats` обеспечивает автоматическое обновление витрины `player_season_stats` после каждой пакетной вставки в таблицу фактов. Определяет `season_id` по идентификатору последней вставленной игры и передаёт его в процедуру `update_season_stats`. Если `season_id` не удаётся определить (таблица пуста), функция завершается без вызова процедуры. Триггер срабатывает на уровне оператора (`FOR EACH STATEMENT`), что исключает многократный пересчёт витрины при пакетной загрузке данных. Схема алгоритма представлена в Приложении А (рисунок А.7).

2.3.2 Алгоритм агрегирования командной статистики

Командная статистика вычисляется динамически через представление `v_team_stats`, исключая дублирование данных в отдельной таблице. Для каждой пары «команда – сезон» представление суммирует показатели всех игроков из `game_player_stats` по матчам, в которых данная команда выступала в качестве хозяев или гостей, и делит накопленные суммы на число матчей. Принадлежность строки статистики к хозяевам или гостям определяется через `JOIN` с таблицей `games` по полям `home_team_id` и `away_team_id`. Результирующий набор содержит средние очки, подборки, передачи, потери и эффективные показатели команды за матч.

2.3.3 Алгоритм массового обновления агрегатов

Процедура `update_season_stats` пересчитывает витрину `player_season_stats` для указанного сезона. Алгоритм спроектирован с отказом от курсоров: применяется один наборочный `INSERT ... SELECT` с последующим `ON CONFLICT DO UPDATE`. Логика процедуры включает пять шагов:

- 1) агрегация по (`player_id`, `team_id`) из `game_player_stats` за сезон: средние показатели, проценты бросков, отбор строк с $\sum MP \geq 50$;
 - 2) для каждой квалифицированной строки вызов `calculate_usg`, внутри которого командные `TmFGA`, `TmFTA`, `TmTOV` и `TmMP` суммируются по всем игрокам команды в матчах, где участвовал данный игрок (фильтр по `game_id` и `team_id`);
 - 3) для `calculate_per` и `calculate_bpm` — двухпроходный расчёт: сначала вычисляется ненормализованный показатель игрока, затем среднее по лиге за сезон (только игроки с $\sum MP > 100$), после чего результат нормируется относительно лигового среднего;
 - 4) вызов `calculate_efg` и `calculate_ts` по сезонным суммам игрока;
 - 5) `UPSERT` в `player_season_stats` по ключу (`player_id`, `season_id`, `team_id`).
- Схема алгоритма хранимой процедуры представлена в Приложении А (рисунок А.8).

2.4 Проектирование ролевой модели доступа

В базе данных определены три роли: `db_admin`, `analyst` и `reader`. Администратору выданы полные права на таблицы, последовательности и функции схемы. Аналитику доступны выборка из таблиц и выполнение расчётных функций. Читатель работает исключительно через представления без прямого доступа к базовым таблицам. Создание и изменение структуры объектов базы данных выполняются от имени выделенного пользователя-владельца схемы при развёртывании. Ограничение доступа роли `reader` к полному представлению `v_player_rankings` и выдача прав только на `v_top_players` обусловлены бизнес-логикой: публичным пользователям доступна лишь краткая сводка лидеров, в то время как полные аналитические выгрузки предназначены только для аналитиков.

Таблица 2.9 — Матрица прав доступа к объектам базы данных

Объект	db_admin	analyst	reader
positions, seasons, teams, players	CRUD	SELECT	—
games, game_player_stats	CRUD	SELECT	—
player_season_stats, player_team_history	CRUD	SELECT	—
Функции calculate_*	EXECUTE	EXECUTE	—
v_team_stats, v_top_players	—	SELECT	SELECT
v_player_rankings	SELECT	SELECT	—

2.5 Проектирование программного обеспечения и кэширования

2.5.1 Архитектура приложения

Программный комплекс построен по трёхзвенной архитектуре. Уровень представления формирует HTTP-запросы к серверу. Уровень приложения — REST API-сервер, выполняющий маршрутизацию запросов, проверку авторизации и вызов процедур СУБД. Уровень данных — реляционная СУБД и in-мемори кэш, взаимодействие которых описано в подразделе 2.5.3.

API организован в пять групп маршрутов (`players`, `teams`, `stats`, `league`, `admin`), соответствующих вариантам использования из раздела 1.5.2. Итого спроектировано 20 бизнес-маршрутов.

Компонентная диаграмма трёхзвенной архитектуры приведена на рисунке 2.2. Уровень представления спроектирован в виде одностраничного веб-приложения (SPA); уровень приложения — асинхронный REST API сервер; уровень данных — реляционная СУБД, дополненная in-мемори хранилищем формата «ключ-значение».

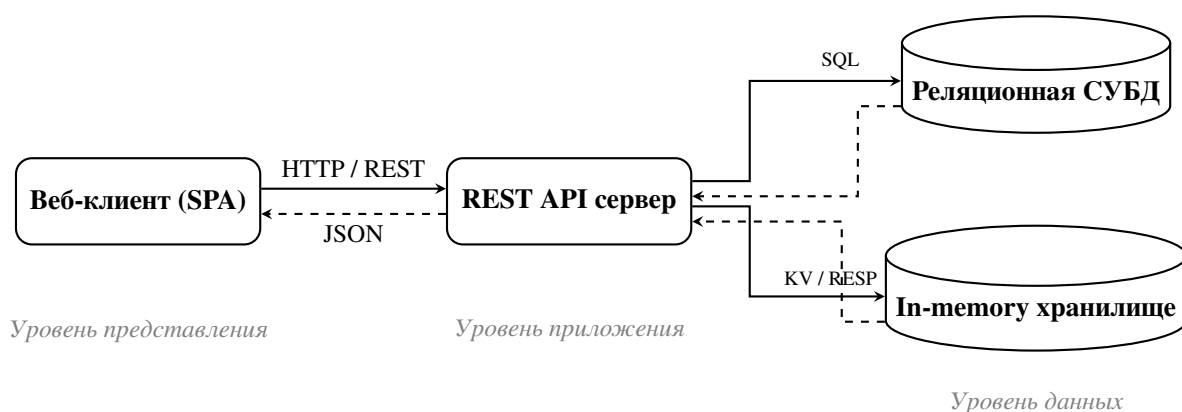


Рисунок 2.2 — Компонентная диаграмма трёхзвенной архитектуры системы

2.5.2 Спецификация REST API

Сезон в параметрах запроса передаётся целочисленным `season_id` (внешний ключ таблицы `seasons`); метка `label`, в частности формата 2023-24, возвращается в ответах API для отображения в интерфейсе. Список сезонов клиент получает через GET `/league/seasons`.

Таблица 2.10 — Маршруты группы `/players`

Метод	Путь	Параметры	Вариант исп.
GET	<code>/players</code>	<code>season_id</code> (обяз.), <code>position</code> , <code>team_id</code> , <code>search</code> , <code>limit</code> , <code>offset</code>	Просмотр статистики
GET	<code>/players/{player_id}</code>	—	Просмотр статистики
GET	<code>/players/{player_id}/stats</code>	<code>season_id</code> (опц.)	Просмотр статистики
GET	<code>/players/{player_id}/gamelog</code>	<code>season_id</code> (обяз.)	Просмотр статистики
GET	<code>/players/{player_id}/career</code>	—	Просмотр статистики
GET	<code>/players/{player_id}/teams</code>	—	Просмотр статистики

Таблица 2.11 — Маршруты группы `/teams`

Метод	Путь	Параметры	Вариант исп.
GET	<code>/teams</code>	—	Просмотр статистики
GET	<code>/teams/{team_id}</code>	—	Просмотр статистики
GET	<code>/teams/{team_id}/roster</code>	<code>season_id</code> (обяз.)	Просмотр статистики
GET	<code>/teams/{team_id}/games</code>	<code>season_id</code> (опц.)	Просмотр статистики

Таблица 2.12 — Маршруты группы /stats

Метод	Путь	Параметры	Вариант исп.
GET	/stats/leaders	metric, season_id (обяз.), team_id, position, min_games, limit	Просмотр статистики
GET	/stats/advanced	season_id (обяз.)	Расчёт метрик
GET	/stats/scatter	season_id (обяз.)	Расчёт метрик
GET	/stats/boxscore/{game_id}	—	Просмотр статистики
GET	/stats/compare/players	p1_id, p2_id, season_id (обяз.)	Просмотр статистики

Параметр `metric` в маршруте `/stats/leaders` принимает одно из значений: `avg_pts`, `avg_reb`, `avg_ast`, `avg_stl`, `avg_blk`, `per`, `ts_pct`, `efg_pct`, `bpm` и `usg_pct`.

Таблица 2.13 — Маршруты группы /league

Метод	Путь	Параметры	Вариант исп.
GET	/league/seasons	—	Просмотр статистики
GET	/league/dashboard	season_id (обяз.)	Просмотр статистики
GET	/league/trends	—	Просмотр статистики
GET	/league/search	q (обяз., мин. 2 символа)	Просмотр статистики

Маршрут `/league/seasons` возвращает перечень доступных сезонов. Ответ содержит поля `season_id`, `label`, `season_type`, `start_date` и `end_date`; клиент использует его для формирования элементов выбора сезона. Маршрут `/league/search` реализует полнотекстовый поиск или поиск по подстроке (LIKE) по полям `first_name` и `last_name` таблицы `players`, а также `name` таблицы `teams`, возвращая комбинированный массив совпадений.

Таблица 2.14 — Маршруты группы /admin

Метод	Путь	Параметры	Вариант исп.
POST	/admin/refresh/{season_id}	заголовок X-API-Key	Управление данными

Разграничение доступа на уровне API реализовано для группы `/admin`: маршрут проверяет наличие корректного значения в заголовке `X-API-Key`. Разграничение ролей `analyst` и `reader` обеспечивается средствами СУБД в соответствии с матрицей прав (таблица 2.9): API-сервер устанавливает соединение с базой данных от имени пользователя с соответствующим набором привилегий.

2.5.3 Кэширование результатов запросов

Высокая пропускная способность обеспечивается in-memory хранилищем, реализующим паттерн сквозного кэширования с ленивой записью.

Алгоритм: приложению поступает REST-запрос. Сервер проверяет наличие ключа в кэше. При попадании данные возвращаются из кэша. При промахе сервер запрашивает реляционную БД, сериализует ответ, сохраняет его в кэш с заданным временем жизни и возвращает клиенту.

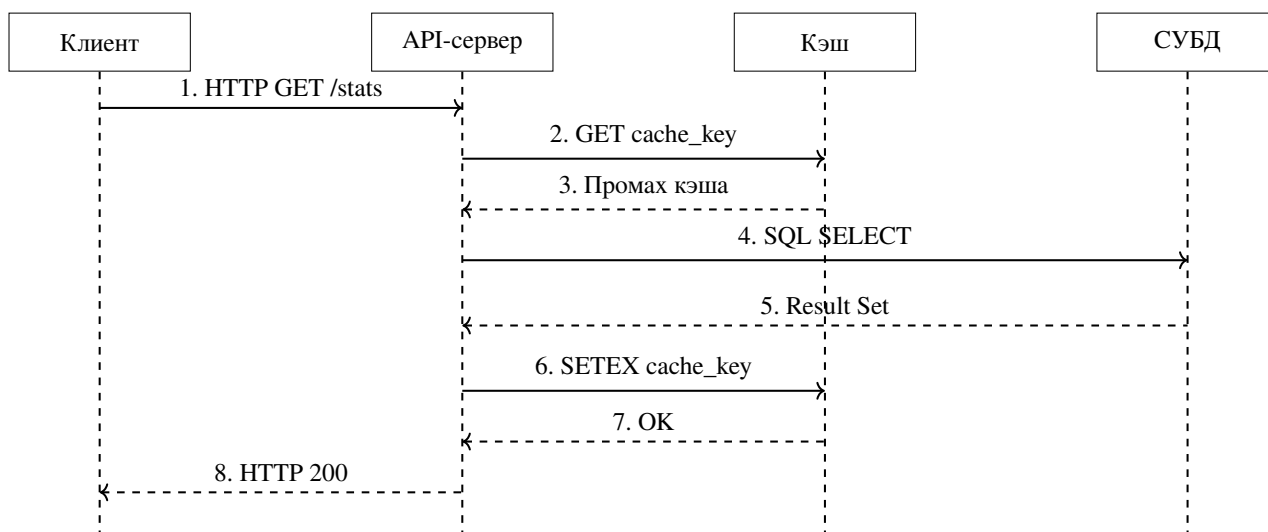


Рисунок 2.3 — Диаграмма последовательности при паттерне Cache-Aside

Инвалидация кэша выполняется по шаблонам ключей лидербордов, дашборда и данных команд только для изменённого сезона, а не полным сбросом кэша: это предотвращает одновременную инвалидацию всех ключей и снижает риск пиковой нагрузки на СУБД при массовом промахе кэша — эффекта «шторма кэша», который возникает, когда множество параллельных запросов одновременно обнаруживают истёкший ключ и нагружают базу данных.

Вывод

Спроектирована реляционная схема из 8 таблиц, 3 представлений и 6 ключевых индексов; показано соответствие всех таблиц 1НФ, 2НФ и 3НФ. Бизнес-логика метрик реализована SQL-процедурами на стороне СУБД; ограничения целостности закреплены на уровне CHECK-ограничений и триггера. Определена ролевая модель из трёх ролей с разграничением прав на уровне таблиц, функций и представлений. Спроектировано REST API из 20 маршрутов, организованных в пять групп и покрывающих все три варианта использования системы. Для снижения задержки аналитических выборок спроектировано кэширование по паттерну Cache-Aside с фиксированной инвалидацией.

3 Технологический раздел

3.1 Выбор и обоснование средств реализации

Профиль нагрузки проектируемой системы является гибридным (HTAP): преобладают аналитические запросы на чтение с агрегацией, при этом процедура массового обновления статистики требует транзакционной целостности.

3.1.1 СУБД

В качестве основной СУБД выбрана PostgreSQL 15 [17] в связи с наличием проверенных драйверов асинхронного взаимодействия и стабильной работой используемых механизмов процедурного расширения. Выбор данного решения обоснован следующими факторами:

- наличие встроенного процедурного языка PL/pgSQL, необходимого для реализации сложных математических алгоритмов расчёта продвинутых спортивных метрик на стороне базы данных;
- поддержка атомарных операций слияния строк через конструкцию пакетного обновления для эффективного пересчёта сезонных агрегатов без возникновения состояния гонки;
- наличие развитого оптимизатора запросов для построения эффективных планов выполнения с использованием B-tree индексов при многотабличных операциях соединения;
- возможность декларативного обеспечения логической целостности данных с помощью проверочных ограничений и механизма триггеров на уровне схемы данных.

3.1.2 Кэширование

Паттерн Cache-Aside реализован на базе in-memory СУБД Redis 7 [18]. Выбор обоснован поддержкой операций сканирования и группового удаления по шаблону, что необходимо для инвалидации связанных ключей при пересчёте статистики отдельного сезона. Команда SETEX позволяет атомарно устанавливать значение и время жизни ключа, исключая состояние гонки.

3.1.3 Серверный слой

REST API реализован на Python 3.11 с фреймворком FastAPI [19]. Фреймворк построен на стандарте ASGI и модели асинхронного ввода-вывода: каждый обработчик маршрута является корутиной, что позволяет обслуживать конкурентные запросы в одном процессе без блокировки потока. Механизм внедрения зависимостей позволяет декларативно привязать к маршруту сессию конкретной роли СУБД, что соответствует ролевой модели из раздела 2.4: разграничение привилегий происходит на уровне соединения с PostgreSQL, а не в логике приложения. Для взаимодействия с СУБД применяется драйвер asyncpg [20], реализующий бинарный протокол PostgreSQL, в связке с SQLAlchemy 2.0 для конструирования динамических аналитических

запросов.

3.1.4 Клиентский слой

Клиентская часть представляет собой одностраничное веб-приложение на базе библиотеки React 18 [21] и строго типизированного языка TypeScript. Выбор данного стека технологий обусловлен компонентным подходом к разработке пользовательского интерфейса, что позволяет эффективно переиспользовать элементы отображения: таблицы статистики, графики и карточки игроков. Использование механизма теневого дерева документа в React обеспечивает высокую производительность при рендеринге объёмных наборов данных и динамическом обновлении аналитических дашбордов без полной перезагрузки страницы.

3.2 Физическая реализация базы данных

3.2.1 Создание таблиц и ограничений

Физическая реализация структуры базы данных выполнена с помощью DDL-скриптов (полный листинг приведён в Приложении Б). В листинге 3.1 представлен фрагмент создания таблицы фактов `game_player_stats`, иллюстрирующий механизм декларативного обеспечения ссылочной и логической целостности.

Листинг 3.1 — DDL таблицы `game_player_stats` (фрагмент)

```
CREATE TABLE game_player_stats (  
    stat_id    SERIAL PRIMARY KEY,  
    game_id    INTEGER NOT NULL  
                REFERENCES games(game_id) ON DELETE CASCADE,  
    player_id  INTEGER NOT NULL  
                REFERENCES players(player_id) ON DELETE RESTRICT,  
    team_id    INTEGER NOT NULL  
                REFERENCES teams(team_id) ON DELETE RESTRICT,  
    points     SMALLINT NOT NULL DEFAULT 0 CHECK (points >= 0),  
    fouls      SMALLINT NOT NULL DEFAULT 0 CHECK (fouls BETWEEN 0  
AND 6),  
    CONSTRAINT uq_game_player UNIQUE (game_id, player_id)  
);
```

3.2.2 Реализация индексов

Аналитические индексы (Приложение Б) реализованы на базе структуры В-дерева. В листинге 3.2 продемонстрировано создание ключевых индексов для таблицы фактов.

Листинг 3.2 — Создание индексов аналитических запросов (фрагмент)

```
CREATE INDEX idx_gps_player_id
    ON game_player_stats(player_id);
CREATE INDEX idx_gps_team_game
    ON game_player_stats(team_id, game_id);
CREATE INDEX idx_pss_season
    ON player_season_stats(season_id, per DESC NULLS LAST);
```

Исследование эффективности данных структур и их влияния на время выполнения запросов приведено в исследовательском разделе (глава 4).

3.2.3 Загрузка тестовых данных

Первоначальное наполнение базы данных выполнено средствами языка Python через официальный API NBA Stats [1] и библиотеку `nba_api` [22], формирующую HTTP-запросы к сервису `stats.nba.com`. Скрипт извлекает архивную статистику NBA за пять регулярных сезонов 2019/20–2023/24.

Полученные данные нормализуются и загружаются в реляционные таблицы PostgreSQL пакетами в порядке, соответствующем иерархии внешних ключей: справочники, затем игроки, матчи, статистика матчей, история команд. Использование официального источника сохраняет репрезентативное распределение показателей реальных игроков, что критически важно для корректной верификации относительных метрик.

По результатам контрольных запросов `SELECT COUNT(*)` объём ключевых таблиц после загрузки приведён в таблице 3.1. Таблица фактов `game_player_stats` содержит 150 629 записей, таблица `games` — 5 829 матчей. Это полностью покрывает нефункциональное требование об объёме данных.

Таблица 3.1 — Объём загруженных данных (контрольный запрос `SELECT COUNT(*)`)

Таблица	Число строк
<code>game_player_stats</code>	150 629
<code>games</code>	5 829
<code>players</code>	5 126
<code>player_season_stats</code>	2 813
<code>teams</code>	30
<code>positions</code>	5
<code>seasons</code>	5

3.2.4 Примеры запросов к базе данных

Для верификации корректности схемы и демонстрации аналитических возможностей системы разработан набор SQL-запросов, охватывающий основные классы операций выборки.

Простой запрос с фильтрацией. Выборка активных игроков позиции центрального (текст запроса см. в Приложении В, листинг В.1). Результат выполнения запроса приведён в таблице 3.2.

Таблица 3.2 — Результат запроса с фильтрацией по позиции центрального

ID	Имя	Фамилия
23	Steven	Adams
25	Bam	Adebayo
61	Jarrett	Allen
172	Deandre	Ayton
260	Charles	Bassey
361	Goga	Bitadze
363	Bismack	Biyombo
480	Tony	Bradley
602	Thomas	Bryant
694	Clint	Capela

Многотабличный запрос. Получение результатов матчей с названиями команд и меткой сезона (текст запроса см. в Приложении В, листинг В.2). Результат выполнения запроса (первые 10 строк из 1230) приведён в таблице 3.3.

Таблица 3.3 — Результат многотабличного запроса по матчам сезона 2023-24

Сезон	Дата	Хозяева	Дом.	Гост.	Гости	ОТ
2023-24	24.10.2023	Denver Nuggets	119	107	Los Angeles Lakers	0
2023-24	24.10.2023	Golden State Warriors	97	108	Phoenix Suns	0
2023-24	25.10.2023	Los Angeles Clippers	123	111	Portland Trail Blazers	0
2023-24	25.10.2023	Miami Heat	103	102	Detroit Pistons	0
2023-24	25.10.2023	Orlando Magic	116	86	Houston Rockets	0
2023-24	25.10.2023	Charlotte Hornets	116	110	Atlanta Hawks	0
2023-24	25.10.2023	Indiana Pacers	143	120	Washington Wizards	0
2023-24	25.10.2023	Chicago Bulls	104	124	Oklahoma City Thunder	0
2023-24	25.10.2023	Toronto Raptors	97	94	Minnesota Timberwolves	0
2023-24	25.10.2023	Utah Jazz	114	130	Sacramento Kings	0

Итоговый запрос с агрегацией и группировкой. Средняя результативность по позициям за сезон, с фильтрацией игроков с достаточным игровым временем (текст запроса см. в Приложении В, листинг В.3). Результат выполнения запроса приведён в таблице 3.4.

Таблица 3.4 — Средняя результативность по позициям (сезон 2023-24)

Поз.	Игроков	PTS	REB	AST	PER
C	35	12.74	8.32	2.17	21.40
PF	19	13.63	6.86	2.51	20.93
SG	129	13.20	3.53	3.84	15.67
SF	110	11.53	4.42	2.29	14.96

Запрос с подзапросом. Игроки, чей PER превышает среднее по лиге за сезон (текст запроса см. в Приложении В, листинг В.4). Результат выполнения запроса (10 строк из 271) приведён в таблице 3.5.

Таблица 3.5 — Игроки с PER выше среднего по лиге (сезон 2023-24)

Игрок	Команда	PER	PTS
Joel Embiid	PHI	37.60	31.47
Nikola Jokić	DEN	36.41	26.39
Giannis Antetokounmpo	MIL	34.85	30.44
Luka Dončić	DAL	32.90	33.86
Anthony Davis	LAL	30.96	23.75
Shai Gilgeous-Alexander	OKC	30.77	30.05
Leonard Miller	MIN	30.75	0.91
Victor Wembanyama	SAS	29.65	21.14
Domantas Sabonis	SAC	29.41	19.43
LeBron James	LAL	28.98	24.96

Составной запрос (UNION). Сводный список лидеров по очкам и передачам в одной выборке (текст запроса см. в Приложении В, листинг В.5). Результат выполнения запроса приведён в таблице 3.6.

Таблица 3.6 — Топ-5 лидеров по очкам и передачам (сезон 2023-24)

Категория	Игрок	Значение
Scoring	Luka Dončić	33.86
Scoring	Joel Embiid	31.47
Scoring	Giannis Antetokounmpo	30.44
Scoring	Shai Gilgeous-Alexander	30.05
Scoring	Devin Booker	27.07
Assists	Tyrese Haliburton	10.90
Assists	Luka Dončić	9.80
Assists	Trae Young	9.40
Assists	Nikola Jokić	8.96
Assists	James Harden	8.53

Приведённые запросы охватывают простую фильтрацию, многотабличные соединения, агрегацию с группировкой, скалярные подзапросы и составные запросы с объединением результатов.

3.3 Реализация логики предметной области в СУБД

3.3.1 Хранимые функции расчёта метрик

Логика расчёта спортивных метрик реализована в виде хранимых PL/pgSQL-функций (Приложение Г). Функция `calculate_ts` включает программную защиту от деления на ноль (Приложение Г, листинг Г.1).

Все расчётные функции помечены квалификатором `STABLE`, что позволяет оптимизатору запросов кэшировать результаты вызовов в рамках одного оператора. Агрегация сезонной статистики выполняется хранимой процедурой `update_season_stats` с использованием атомарной конструкции `INSERT ... ON CONFLICT DO UPDATE`.

3.3.2 Триггеры логической целостности

Для защиты данных от аномалий реализован триггер `trg_check_team_score`. Он проверяет бизнес-правило предметной области: сумма очков всех игроков команды в конкретном матче должна равняться итоговому командному счёту. Текст функции-триггера приведён в Приложении Г (листинг Г.2).

3.4 Реализация ролевой модели доступа

Настройка прав доступа выполнена в соответствии с принципом минимальных привилегий (Приложение Г, листинг Г.3). Роль публичного доступа (`reader`) лишена возможности прямого обращения к базовым таблицам; чтение осуществляется исключительно через защищённые представления.

Корректность ограничений для роли `reader` проверена подключением к БД от имени пользователя с назначенной ролью `reader`: запрос `SELECT * FROM game_player_stats` завершился ошибкой `permission denied` на уровне PostgreSQL, тогда как `SELECT` из представлений `v_team_stats` и `v_top_players`, перечисленных в приложении Г (листинг Г.3), выполняется успешно. Аналитические маршруты API, в том числе `GET /teams/standings`, открывают сессию под ролью `analyst` и для сводной командной статистики обращаются к представлению `v_team_stats`. У роли `analyst` есть `SELECT` к базовым таблицам (табл. 2.9), поэтому представления для неё служат удобными витринами, а не единственным способом чтения данных.

3.5 Реализация программного обеспечения и интерфейса

3.5.1 Серверное приложение

Серверное приложение реализовано в соответствии со спецификацией REST API конструкторского раздела (подраздел 2.5.2). Точкой входа служит `app/main.py`, регистрирующий пять групп маршрутов и CORS-middleware. Подключение к PostgreSQL организовано через три

независимых пула соединений — по одному на каждую роль СУБД. Привязка маршрута к роли осуществляется через аргумент Depends:

Листинг 3.3 — Привязка маршрута к роли СУБД

```
@router.get("/leaders")
async def get_leaders(
    season_id: int,
    db: AsyncSession = Depends(get_db_analyst),
    cache: CacheManager = Depends(get_cache),
):
```

Маршрут `/stats/leaders` открывает соединение от имени роли `analyst`, имеющей право `SELECT` к агрегированной статистике и `EXECUTE` к функциям расчёта метрик; прямое изменение базовых таблиц по-прежнему запрещено. Жизненный цикл сессии управляется генератором: при любом исходе обработчика сессия закрывается, а незафиксированные изменения откатываются.

3.5.2 Кэширование

Алгоритм паттерна `Cache-Aside` (класс `CacheManager`): генерация ключа на основе параметров, чтение из `Redis`; при промахе — выполнение SQL-запроса, кэширование результата с установкой времени жизни (TTL) и возврат ответа. Пример приведён в листинге 3.4.

Листинг 3.4 — `Cache-Aside` в маршруте `/stats/leaders`

```
cache_key = (
    f"leaders:{metric}:{season_id}:{team_id}:{position}:{min_games}"
    f":{limit}"
)
cached = await cache.get(cache_key)
if cached:
    return cached
result = await db.execute(query)
data = [...]
await cache.set(cache_key, data, ttl=900)
return data
```

Время жизни ключей дифференцировано по типу данных: данные завершённых матчей кэшируются на один год, лидерборды и списки составов команд — на 15 минут, результаты поиска — на 2 минуты. Инвалидация при вызове `POST /admin/refresh/{season_id}` выполняется по шаблонам ключей затронутого сезона в соответствии с проектным решением подраздела 2.5.3.

3.5.3 Пользовательский интерфейс

Интеграция библиотеки Recharts [23] обеспечивает рендеринг диаграмм и графиков трендов.

Навигация включает аналитический дашборд с KPI-метриками лиги, таблицу игроков с серверной фильтрацией, турнирные сводки конференций, глобальный рейтинг лидеров и раздел расширенной статистики с диаграммами рассеяния.

Модуль сравнения игроков реализован в виде интерактивного модального окна с совмещённым радарным графиком, гистограммой базовой статистики и цветовым выделением превосходящих метрик.

Внешний вид раздела Dashboard, а также модальных окон профиля и сравнения игроков представлен в Приложении Е.

3.6 Тестирование и верификация аналитических данных

Тестирование логики СУБД проводилось методом чёрного ящика. Для проверки точности математических агрегаций выполнен регрессионный тест на реальных данных из загруженной базы: эталонные значения получены независимым расчётом по формулам раздела 1.4, после чего сопоставлены с результатами хранимых функций PostgreSQL.

В качестве тестового объекта выбран профиль игрока Nikola Jokić с идентификатором 2327 за регулярный сезон 2023-24 с идентификатором 5. Прямой SQL-запрос суммы сырых фактов из таблицы `game_player_stats` вернул следующие значения:

- очки: 2085;
- броски с игры: 1411;
- штрафные попытки: 438.

Ручной расчёт истинного процента бросков по формуле (1.2):

$$TS\% = \frac{2085}{2 \cdot (1411 + 0,44 \cdot 438)} = \frac{2085}{3207,44} \approx 0,6501.$$

Для верификации в СУБД выполнен вызов хранимой функции:

```
SELECT calculate_ts(2327, 5);
```

Результат совпал с ручным расчётом. Аналогичная верификация проведена для `eFG%` и `USG%`; расхождений не выявлено.

Верификация `PER` выполнена по той же методике. Из таблицы `game_player_stats` получены сезонные суммы для `player_id = 2327`: `PTS = 2085`, `TRB = 976`, `AST = 708`, `STL = 108`, `BLK = 68`, `TOV = 237`, `PF = 194`, `FGM/FGA = 822/1411`, `FTM/FTA = 358/438`, `MP = 2736,4`. Ненормализованный вклад по формуле `uPER` (раздел 1.4):

$$uPER = (2085 + 976 + 708 + 108 + 68 - 237 - 194 - 589 - 35,2) \cdot \frac{48}{2736,4} = 2889,8 \cdot \frac{48}{2736,4} \approx 50,69.$$

Лиговое среднее по формуле (1.5) по 475 игрокам с $MP > 100$: $lg_aPER = 20,88$. Итоговый рейтинг по формуле (1.6):

$$PER = 50,69 \cdot \frac{15}{20,88} \approx 36,41.$$

Вызов `SELECT calculate_per(2327, 5)` вернул 36,41 — расхождений не выявлено.

Тестирование граничных случаев подтвердило корректный возврат значения `NULL` при отсутствии попыток бросков и нулевом времени на площадке. Работа триггера `trg_check_team_score` проверена попыткой вставки строки с намеренно неверной суммой очков: триггер поднял исключение `Player points sum does not match team score`, вставка была отклонена — поведение соответствует ожидаемому (таблица 3.7).

Таблица 3.7 — Контрольные тест-кейсы функций расчёта метрик

№	Сценарий	Входные данные	Ожидание	Результат
1	Нормальный профиль	<code>calculate_ts(2327, 5)</code>	$\approx 0,6501$	совпало
2	Нет попыток броска	<code>calculate_ts(0, 0)</code>	<code>NULL</code>	<code>NULL</code>
3	Нулевые FGA ($eFG\%$)	<code>calculate_efg(0, 0)</code>	<code>NULL</code>	<code>NULL</code>
4	Пересчёт витрины	<code>CALL update_season_stats(5)</code>	обновлены строки <code>pss</code>	выполнено
5	Верификация PER	<code>calculate_per(2327, 5)</code>	$\approx 36,41$ (формулы 1.4–1.6)	36,41, совпало
6	Триггер целостности счёта	<code>INSERT</code> с суммой очков, не совпадающей с <code>home_score</code>	<code>EXCEPTION</code>	исключение поднято, вставка отклонена

Для формализации тестового покрытия каждой функции на стороне СУБД выделены классы эквивалентности — непересекающиеся разделы входных данных, для которых ожидается одинаковое поведение функции. Разбиение приведено в таблице 3.8.

Таблица 3.8 — Классы эквивалентности функций на стороне СУБД

Функция	КЭ	Условие раздела	Представитель	Ожидание
calculate_efg	КЭ1 КЭ2 КЭ3	FGA > 0, FG3M ≥ 0 FGA = 0 (нет бросков) team_id IS NULL (агрегация по всем командам)	calc_efg(2327, 5) calc_efg(0, 0) calc_efg(2327, 5, NULL)	∈ (0; 1] NULL = КЭ1
calculate_usg	КЭ1 КЭ2 КЭ3 КЭ4	MP ≥ 1, denom > 0 MP < 1 (игрок не выходил) командный знаменатель = 0 team_id не определён (нет матчей)	calc_usg(2327, 5) несуществующий player_id аномальные тестовые данные несуществующий player_id	∈ (0; 1] NULL NULL NULL
calculate_ts	КЭ1 КЭ2 КЭ3	FGA > 0 или FTA > 0 FGA = 0 и FTA = 0 знаменатель = 0 при FGA = 0, FTA > 0 (редкий)	calc_ts(2327, 5) calc_ts(0, 0) аномальные тестовые данные	≈ 0,6501 NULL NULL
calculate_per	КЭ1 КЭ2 КЭ3	MP ≥ 50, lg_aPER > 0 MP < 50 (дисквалифицирован) lg_aPER = 0 или NULL (пустой сезон)	calc_per(2327, 5) игрок с малым числом игр фиктивный season_id без данных	≈ 36,41 NULL нормировка на 15,0
calculate_bpm	КЭ1 КЭ2	MP ≥ 50 MP < 50	calc_bpm(2327, 5) игрок с малым числом игр	числовое значение NULL
trg_validate_team_score_row	КЭ1 КЭ2 КЭ3	статус ≠ Finished или счёт NULL статус Finished, ∑ points = счёт статус Finished, ∑ points ≠ счёт	INSERT в незавершённый матч корректный INSERT INSERT с неверной суммой очков	принято без проверки принято EXCEPTION
trigger_update_season_stats	КЭ1 КЭ2	season_id найден season_id IS NULL (пустая таблица)	штатный INSERT в game_player_stats таблица без строк	вызов update_season_stats возврат NULL без вызова

Вывод

В технологическом разделе обоснован стек технологий (PostgreSQL 15, Redis 7, FastAPI, React 18) и реализована физическая структура базы данных. Разработаны PL/pgSQL-функции

и триггеры, обеспечивающие расчёт метрик и консистентность данных. С помощью написанного ETL-скрипта БД наполнена реальной статистикой NBA (более 150 тыс. фактов). Успешно реализованы механизмы кэширования (Cache-Aside) и ролевого доступа. Регрессионное тестирование на реальных данных подтвердило математическую корректность работы базы данных и хранимых процедур.

4 Исследовательский раздел

4.1 Цель и методика исследования

Целью раздела является количественная оценка влияния объёма данных, стратегий индексации и уровня параллельной нагрузки на производительность аналитических запросов, а также сравнение прямой агрегации по таблице фактов с обращением к предвычисленной витрине и замер эффективности кэширования Redis.

Измерения автоматизированы Python-скриптом, выполняющим прямые SQL-запросы и HTTP-обращения к API. Стенд: СУБД PostgreSQL 15 и API-сервер развёрнуты в изолированных Docker-контейнерах на одном хосте со следующими характеристиками: CPU — Intel Core i5-1235U (10 ядер, 3,3 ГГц), ОЗУ — 16 ГБ DDR4, хранилище — NVMe SSD. Таблица фактов объёмом 150 629 строк, около 35 МБ, полностью помещается в page cache, что исключает дисковый ввод-вывод при повторных запросах и сетевые задержки между контейнерами. Методика включает прогревочные итерации и фиксацию рабочих метрик: медианы, среднего значения и 95-го перцентиля.

4.2 Влияние объёма данных и стратегий индексации

4.2.1 Влияние объёма данных на время выполнения запроса

Время выполнения запроса к витрине для выборки двадцати лучших игроков замерялось с применением составного индекса [17].

Таблица 4.1 — Время выполнения запроса в зависимости от объёма витрины

Объём данных	Строк	Медиана (мс)	Среднее (мс)	P95 (мс)
1 сезон	522	0,21	0,23	0,36
3 сезона	1 681	0,20	0,22	0,39
5 сезонов	2 813	0,23	0,26	0,43

Медиана стабильна в диапазоне 0,20–0,23 мс при росте числа строк в 5,4 раза. Это подтверждает логарифмическую сложность $O(\log N)$ B-tree индекса: выборка топ-20 обходит $O(\log N + 20)$ узлов дерева независимо от общего объёма таблицы.

4.2.2 Исследование стратегий индексации

Сравнивались четыре стратегии индексации на полном объёме витрины.

Листинг 4.1 — Запрос для сравнения стратегий индексирования

```
SELECT player_id, per, avg_pts
FROM player_season_stats
WHERE season_id = 5
```



```
ORDER BY per DESC NULLS LAST
LIMIT 20;
```

Таблица 4.2 — Влияние конфигурации индексов на производительность запроса

Конфигурация индекса	Медиана (мс)	Среднее (мс)	P95 (мс)
1. Без индексов (Seq Scan)	0,51	0,62	1,54
2. B-tree по (season_id)	0,42	0,48	1,03
3. Составной (season_id, per DESC)	0,26	0,28	0,44
4. Частичный индекс (фильтрация пустых значений)	0,51	0,61	1,46

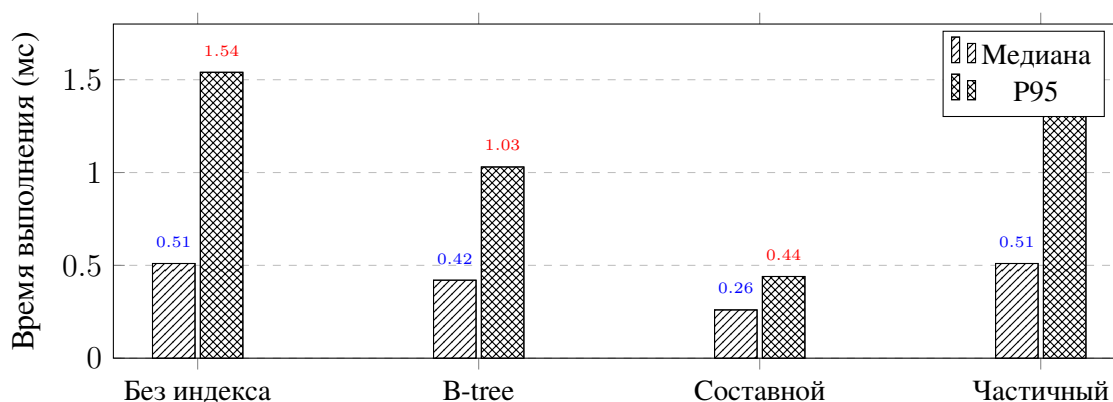


Рисунок 4.1 — Время выполнения запроса при различных стратегиях индексирувания

Простой индекс по идентификатору сезона снижает задержку на 18%, однако требует явной сортировки в памяти. Составной индекс исключает операцию сортировки, улучшая время выполнения на 49%. Частичный индекс игнорируется оптимизатором из-за низкой селективности предиката.

4.3 Сравнение витрины и прямой агрегации по таблице фактов

Третий эксперимент сопоставляет обращение к предвычисленной витрине player_season_stats с прямым агрегирующим запросом к таблице фактов game_player_stats. Объём данных для сезона 5: 32 382 строки фактов, 580 строк витрины.

Тестируемый запрос прямой агрегации:

Листинг 4.2 — Прямой агрегирующий запрос к таблице фактов

```
SELECT gps.player_id ,
       SUM(gps.points)      AS total_pts ,
       COUNT(*)             AS games ,
       AVG(gps.points::float) AS avg_pts
```

```

FROM game_player_stats gps
JOIN games g ON g.game_id = gps.game_id
WHERE g.season_id = 5
GROUP BY gps.player_id
ORDER BY total_pts DESC
LIMIT 20;

```

Таблица 4.3 — Время выполнения прямой агрегации при различных индексах и сравнение с витриной

Конфигурация	Медиана (мс)	Среднее (мс)	P95 (мс)
Прямая агрегация: без индексов	25,45	29,89	55,77
+ idx_games_season_id	35,48	58,42	146,42
+ idx_gps_player_id	22,63	30,39	74,09
+ idx_gps_team_game (оба)	27,56	31,01	64,43
Витрина (составной индекс)	0,37	0,56	0,50

Индекс `idx_gps_player_id` (конфигурация 3) даёт лучший результат прямой агрегации: медиана 22,63 мс. Индекс `idx_games_season_id`, напротив, ухудшает медиану до 35,48 мс относительно запроса без индексов (25,45 мс): планировщик выбирает менее эффективный план `Bitmap Index Scan` вместо хэш-агрегации, поскольку индекс по `season_id` не фильтрует строки достаточно селективно для данного запроса. Одновременное наличие обоих индексов (конфигурация 4) умеренно снижает медиану до 27,56 мс.

Витрина обеспечивает медиану 0,37 мс — в 61 раз быстрее лучшего варианта прямой агрегации (22,63 мс) и в 69 раз быстрее запроса без индексов. Прирост обусловлен исключением `JOIN` и `GROUP BY`: витрина хранит готовые агрегаты, запрос к ней сводится к фильтрации по индексу с прямым чтением результата.

4.4 Исследование эффективности кэширования

Сквозное время ответа API оценено для двух состояний: промах кэша (принудительная инвалидация хранилища перед замером) и попадание в кэш (выборка после прогрева).

Таблица 4.4 — Сравнение времени ответа API: Cache Miss и Cache Hit

Источник данных	Медиана (мс)	Среднее (мс)	P95 (мс)
PostgreSQL (Cache Miss)	26,66	28,58	54,86
Redis (Cache Hit)	7,03	8,33	15,72

Кэширование даёт ускорение в 3,8 раза по медиане (26,66 мс → 7,03 мс). Оба сценария укладываются в нефункциональное требование 100 мс (раздел 1.3.2). Разница обусловлена исключением SQL-выполнения и сериализации: при Cache Hit API выполняет только десериализацию JSON из Redis и HTTP-ответ.

4.5 Исследование производительности при параллельной нагрузке

Деградация производительности при конкурентном доступе оценивалась асинхронным генератором нагрузки на уровнях от 1 до 100 одновременных подключений.

Таблица 4.5 — Зависимость времени ответа API от уровня конкурентной нагрузки

Параллельных запросов	Медиана (мс)	Среднее (мс)	P95 (мс)	Ошибки (%)
1	9,08	8,77	11,56	0
10	35,07	37,42	79,26	0
50	52,94	54,64	90,08	0
100	127,68	128,78	218,92	0

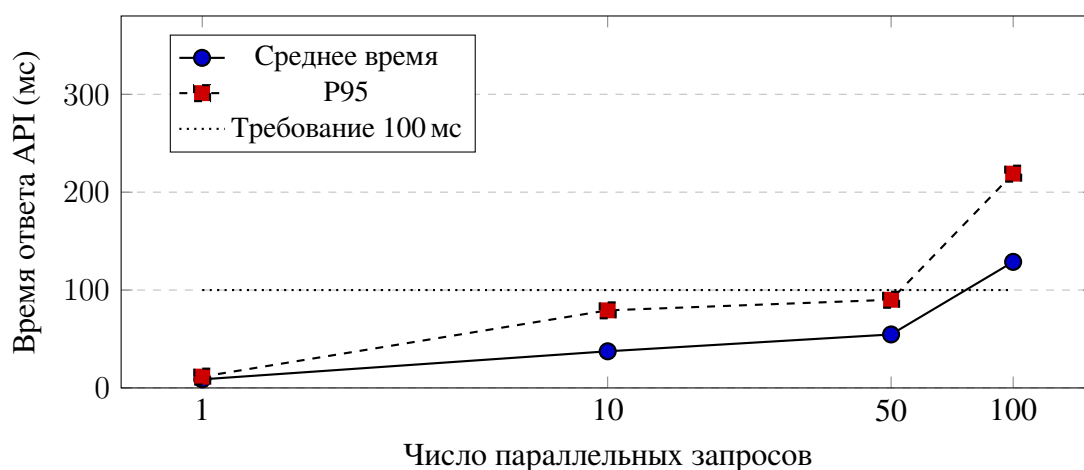


Рисунок 4.2 — Деградация производительности API при росте параллельной нагрузки

При нагрузке до 50 запросов P95 (90,08 мс) укладывается в требование 100 мс. При пиковой нагрузке в 100 подключений показатели времени ответа превышают требования из-за исчерпания пула соединений, однако система сохраняет отказоустойчивость. Деградация обусловлена насыщением пула соединений с PostgreSQL и очереди event loop единственного экземпляра Uvicorn.

4.6 Анализ результатов и рекомендации

1. Витрина как основной инструмент производительности.

Прямая агрегация 32 382 строк фактов с оптимальными индексами занимает 22,63 мс; обращение к витрине — 0,37 мс (ускорение в 61 раз). Предвычисленная витрина `player_season_stats` является ключевым архитектурным решением для выполнения требования 100 мс на аналитических запросах.

2. Составной индекс исключает шаг сортировки.

Составной индекс (`season_id`, `per` DESC) снижает медиану на 49% относительно Seq Scan,

кодируя порядок сортировки в структуре В-дерева. Частичный индекс с неселективным предикатом эквивалентен отсутствию индекса и не рекомендуется. Избыточный составной индекс (`team_id`, `game_id`) при агрегации без предиката по `team_id` ухудшает производительность — планировщик выбирает менее эффективный план; индекс следует применять только при наличии соответствующего предиката в запросе.

3. Кэширование Redis обязательно при конкурентном доступе.

Cache-Aside обеспечивает ускорение в 3,8 раза (26,66 мс → 7,03 мс). При нагрузке 50 запросов без кэша медиана составляет 54 мс с накоплением на СУБД; кэш снижает её до единиц миллисекунд.

4. Предел одного экземпляра — 50 конкурентных запросов.

Система гарантированно выполняет требование $P95 < 100$ мс при нагрузке до 50 подключений. При 100 подключениях оба показателя превышают порог (медиана 127 мс, $P95$ 219 мс). Для пиковых нагрузок рекомендуется горизонтальное масштабирование экземпляров API-сервера за балансировщиком с единым кэшем.

Вывод

По результатам нагрузочных испытаний доказано: предвычисленная витрина `player_season_stats` ускоряет аналитические выборки в 61 раз по сравнению с прямой агрегацией оптимально индексируемых фактов. Составной индекс (`season_id`, `per DESC`) на витрине обеспечивает стабильное время отклика 0,20–0,23 мс при пятикратном росте объёма данных. Кэширование результатов в Redis снижает медианную задержку API в 3,8 раза. Система выполняет требование $P95 < 100$ мс при нагрузке до 50 одновременных запросов; при 100 подключениях оба показателя превышают порог — для таких нагрузок рекомендуется горизонтальное масштабирование API-уровня.

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы была успешно спроектирована и реализована база данных для хранения и анализа спортивной статистики. Поставленная цель достигнута в полном объёме посредством решения сформулированных задач:

- 1) выполнен анализ предметной области. Сформированы строгие критерии к разрабатываемой системе, включающие прозрачность вычислений, наличие программного интерфейса, возможность локального хранения и безопасность. Выделены восемь ключевых сущностей и формализована математическая методология вычисления интегральных метрик, включая эффективный процент с игры, долю владений в атаке и рейтинги эффективности;
- 2) спроектирована реляционная схема данных, приведённая к третьей нормальной форме и состоящая из восьми таблиц. Целостность фактов обеспечена на уровне декларативных ограничений и процедурных триггеров. Реализована ролевая модель на уровне системы управления базами данных, гарантирующая изоляцию аналитических запросов от административных операций;
- 3) разработан серверный слой приложения, предоставляющий сетевой интерфейс из двадцати аналитических маршрутов. Для минимизации задержек спроектирована подсистема кэширования с алгоритмом префиксной инвалидации ключей, предотвращающим лавинообразные нагрузки на базу данных при массовом устаревании кэша;
- 4) выполнена физическая реализация системы. Хранилище наполнено репрезентативной выборкой исторических данных объёмом более ста пятидесяти тысяч фактов. Регрессионное тестирование доказало математическую точность внутрибазовых вычислений: результаты работы хранимых процедур полностью совпали с эталонным независимым расчётом;
- 5) проведено исследование производительности системы при различных сценариях нагрузки. Доказано, что применение денормализованной витрины данных с составным древовидным индексом ускоряет аналитические агрегации в 61 раз. Интеграция подсистемы кэширования снижает медианное время ответа сервера в 3,8 раза. Нагрузочное тестирование подтвердило высокую отказоустойчивость: система выдерживает заданные показатели конкурентного доступа с задержкой менее ста миллисекунд, что полностью удовлетворяет нефункциональным требованиям.

Разработанный программный комплекс представляет собой готовое масштабируемое решение, применимое для обеспечения деятельности аналитических отделов спортивных организаций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NBA Stats [Электронный ресурс]. — Официальная статистика Национальной баскетбольной ассоциации. — Режим доступа: <https://www.nba.com/stats> (дата обращения: 29.03.2026).
2. Kubatko, J. et al. A Starting Point for Analyzing Basketball Statistics // Journal of Quantitative Analysis in Sports. — 2007. — Vol. 3, № 3. — Article 1.
3. Basketball-Reference.com [Электронный ресурс]. — Sports Reference LLC. Справочник статистики NBA и исторические таблицы. — Режим доступа: <https://www.basketball-reference.com/> (дата обращения: 29.03.2026).
4. ESPN NBA [Электронный ресурс]. — Новости и обзоры НБА. — Режим доступа: <https://www.espn.com/nba/> (дата обращения: 29.03.2026).
5. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг ; пер. с англ. — 3-е изд. — М. : Вильямс, 2003. — 1440 с.
6. Player Efficiency Rating [Электронный ресурс] // Basketball-Reference.com. — Описание показателя PER. — Режим доступа: <https://www.basketball-reference.com/about/per.html> (дата обращения: 29.03.2026).
7. Myers, D. About Box Plus/Minus (BPM) [Электронный ресурс] // Basketball-Reference.com. — Режим доступа: <https://www.basketball-reference.com/about/bpm.html> (дата обращения: 29.03.2026).
8. Nielsen, J. Usability Engineering. — San Francisco: Morgan Kaufmann, 1993. — 362 p.
9. Glossary — NBA.com : Stats [Электронный ресурс]. — Термины и определения показателей. — Режим доступа: <https://www.nba.com/stats/help/glossary> (дата обращения: 29.03.2026).
10. Oliver, D. Basketball on Paper: Rules and Tools for Performance Analysis. — Washington, D.C.: Potomac Books, 2004. — 378 p.
11. Hollinger, J. Pro Basketball Forecast. — Washington, D.C.: Potomac Books, 2005. — 432 p.
12. Chen, P. P. The Entity-Relationship Model — Toward a Unified View of Data // ACM Transactions on Database Systems. — 1976. — Vol. 1, № 1. — P. 9–36.

13. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — СПб.: Питер, 2018. — 640 с.
14. Садаладж, П. Дж. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — М.: Вильямс, 2013. — 192 с.
15. ClickHouse Documentation [Электронный ресурс]. — Официальная документация СУБД ClickHouse. — Режим доступа: <https://clickhouse.com/docs/> (дата обращения: 29.03.2026).
16. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; пер. с англ. — 8-е изд. — М. : Вильямс, 2006. — 1328 с.
17. PostgreSQL 15 Documentation [Электронный ресурс]. — Официальная документация СУБД PostgreSQL. — Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 29.03.2026).
18. Redis Documentation [Электронный ресурс]. — Официальная документация in-мемори хранения Redis. — Режим доступа: <https://redis.io/docs/> (дата обращения: 29.03.2026).
19. FastAPI Documentation [Электронный ресурс]. — Официальная документация фреймворка FastAPI. — Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 29.03.2026).
20. asyncpg Documentation [Электронный ресурс]. — Официальная документация асинхронного драйвера PostgreSQL для Python. — Режим доступа: <https://magicstack.github.io/asyncpg/current/> (дата обращения: 29.03.2026).
21. React Documentation [Электронный ресурс]. — Официальная документация библиотеки React. — Режим доступа: <https://react.dev/> (дата обращения: 29.03.2026).
22. nba_api: an API Client Package to Access the APIs of NBA.com [Электронный ресурс]. — Документация Python-библиотеки для доступа к stats.nba.com. — Режим доступа: https://github.com/swar/nba_api (дата обращения: 29.03.2026).
23. Recharts Documentation [Электронный ресурс]. — Официальная документация библиотеки визуализации данных Recharts. — Режим доступа: <https://recharts.org/> (дата обращения: 29.03.2026).

ПРИЛОЖЕНИЯ

Приложение А. Блок-схемы алгоритмов

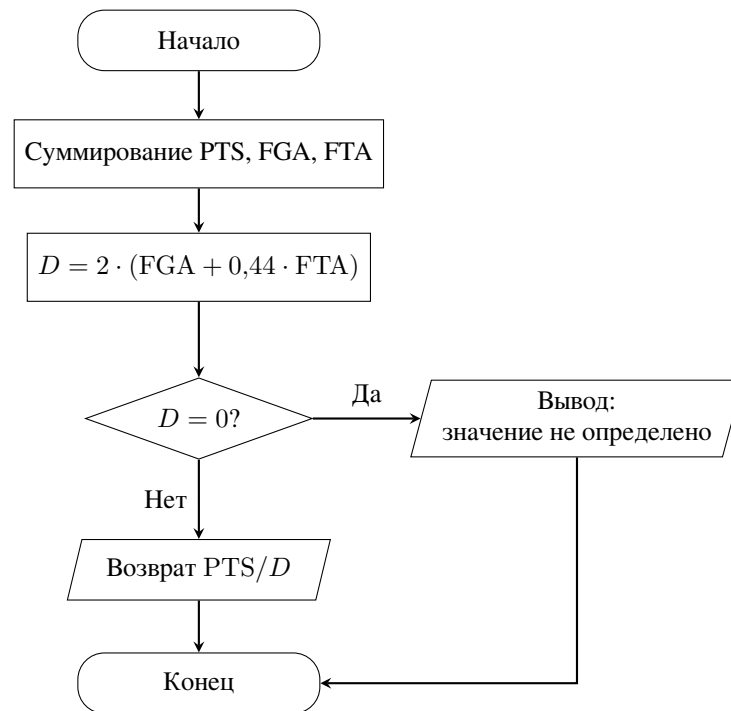


Рисунок А.1 — Схема алгоритма вычисления функции `calculate_ts`

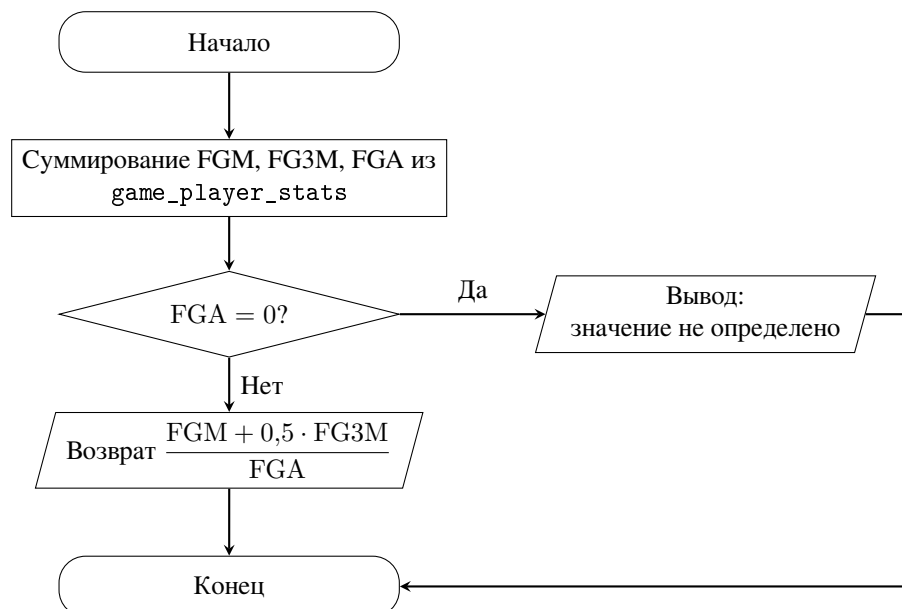


Рисунок А.2 — Схема алгоритма вычисления функции `calculate_efg`

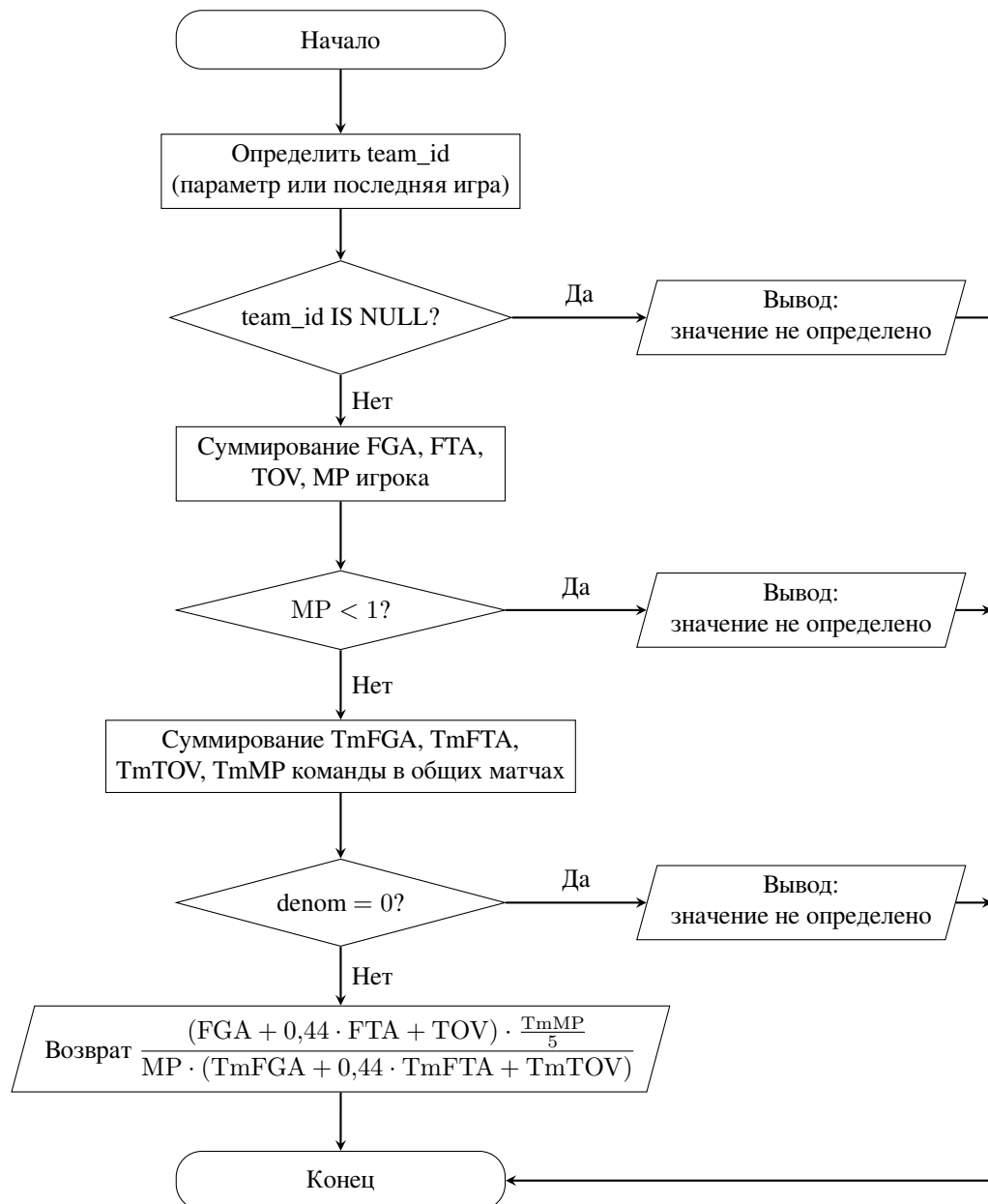


Рисунок А.3 — Схема алгоритма вычисления функции calculate_usg

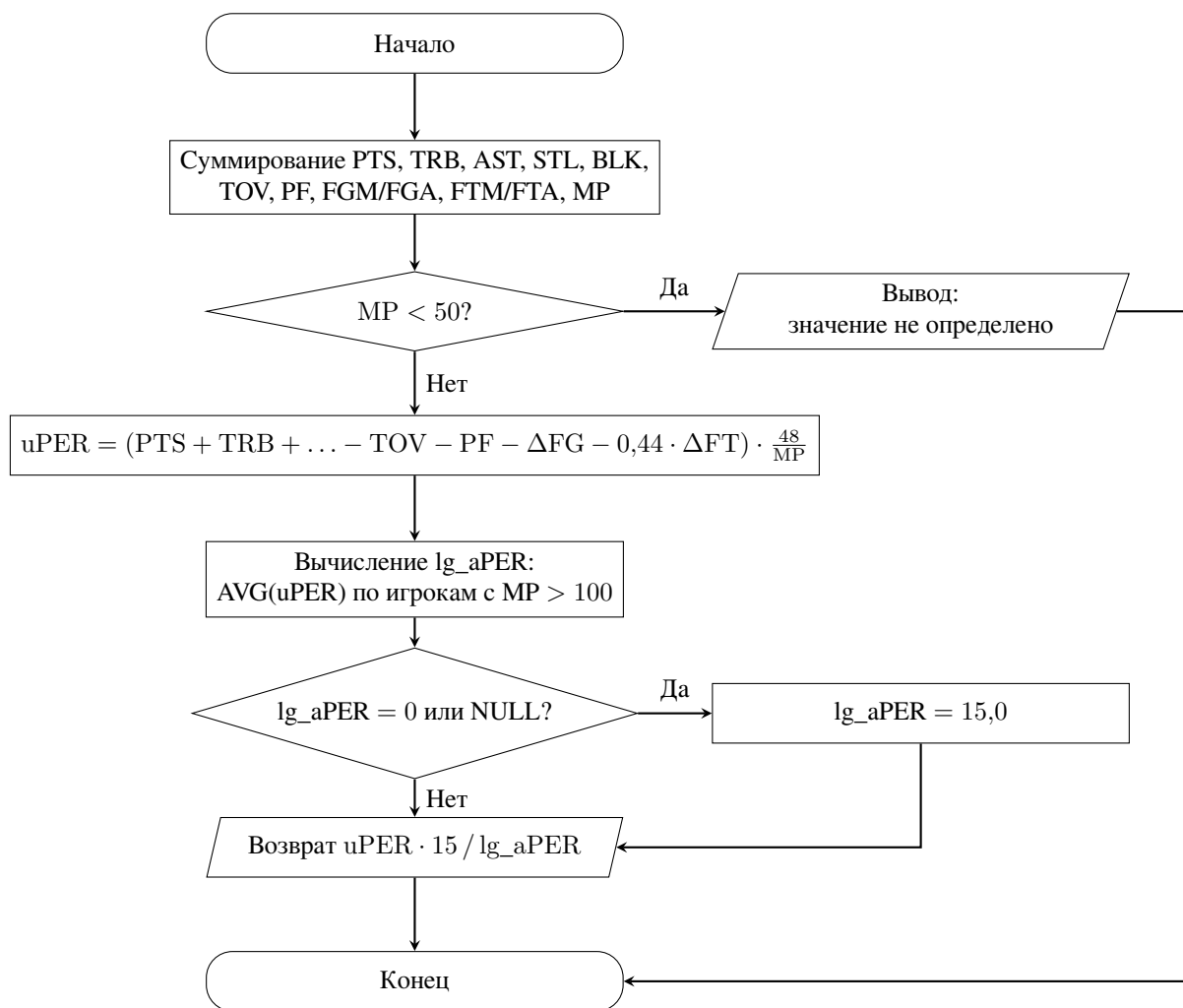


Рисунок А.4 — Схема алгоритма вычисления функции calculate_per

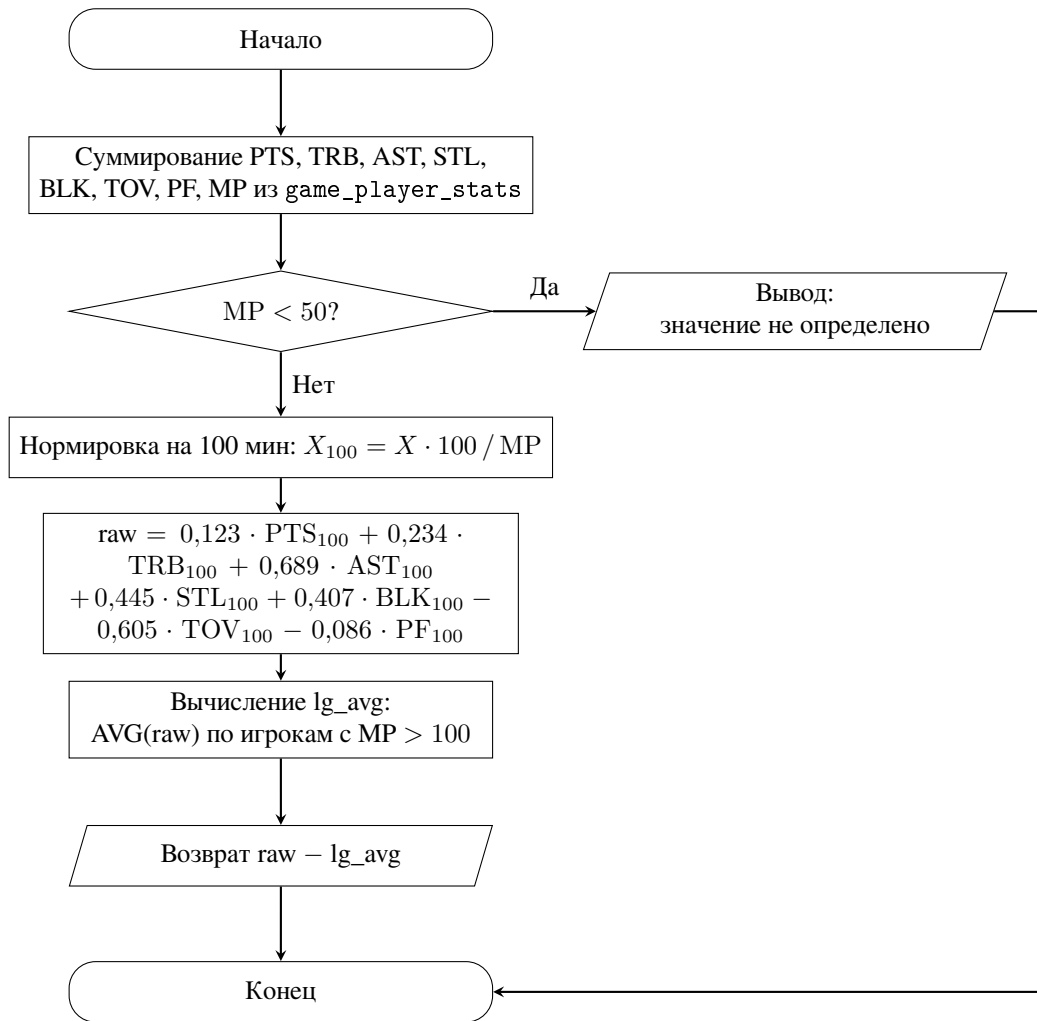


Рисунок А.5 — Схема алгоритма вычисления функции calculate_bpm

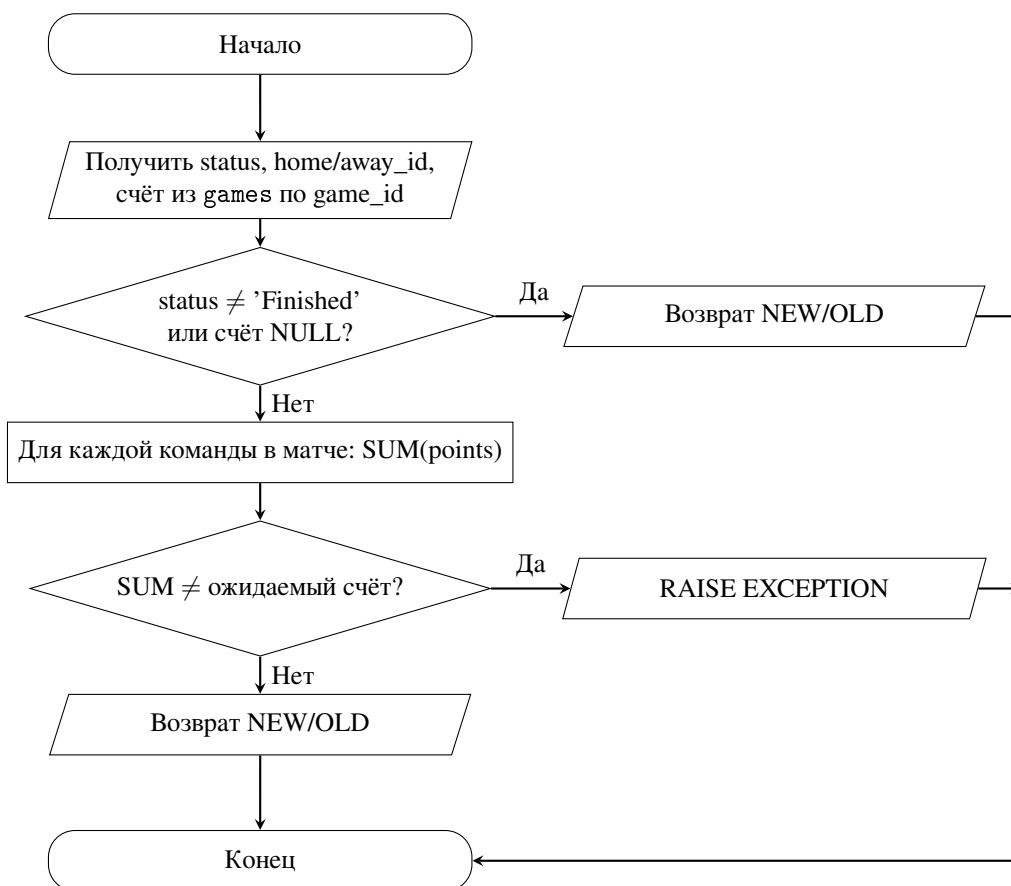


Рисунок А.6 — Схема алгоритма функции-триггера `trg_validate_team_score_row`

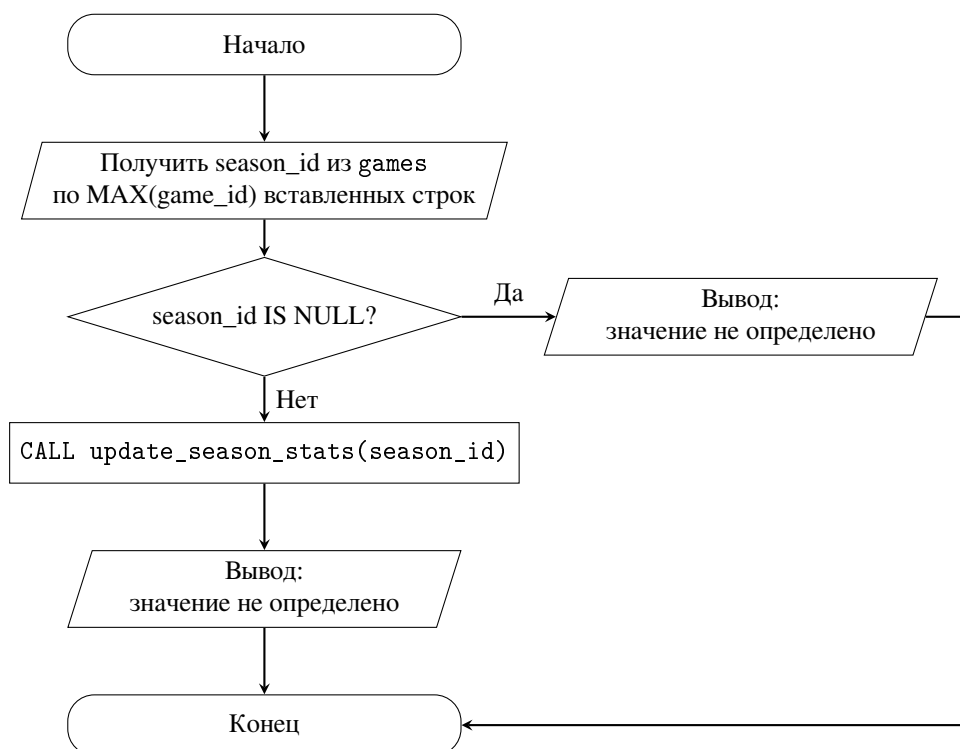


Рисунок А.7 — Схема алгоритма функции-триггера `trigger_update_season_stats`

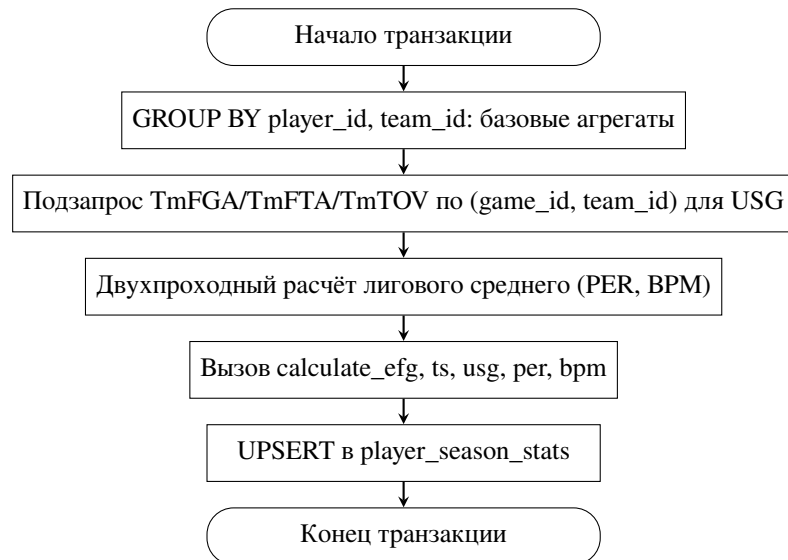


Рисунок А.8 — Схема алгоритма хранимой процедуры update_season_stats

Приложение Б. Скрипты создания структуры базы данных

```

CREATE TABLE positions (
    position_id SERIAL PRIMARY KEY,
    code VARCHAR(2) UNIQUE NOT NULL,
    name VARCHAR(20) NOT NULL
);

CREATE TABLE seasons (
    season_id SERIAL PRIMARY KEY,
    label VARCHAR(9) UNIQUE NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    season_type VARCHAR(15) NOT NULL
);

CREATE TABLE teams (
    team_id SERIAL PRIMARY KEY,
    nba_team_id INT UNIQUE NOT NULL,
    name VARCHAR(60) UNIQUE NOT NULL,
    abbreviation CHAR(3) UNIQUE NOT NULL,
    city VARCHAR(50) NOT NULL,
    conference VARCHAR(4) NOT NULL,
    arena_name VARCHAR(80),
    founded_year SMALLINT,
    is_active BOOLEAN DEFAULT TRUE
);
  
```

Приложение В. Примеры SQL-запросов

Листинг В.1 — Фильтрация игроков по позиции

```
SELECT player_id, first_name, last_name
FROM players
WHERE position_id = (SELECT position_id FROM positions WHERE code =
    'C')
    AND is_active = TRUE
ORDER BY last_name
LIMIT 10;
```

Листинг В.2 — Результаты матчей с JOIN по сезону и командам

```
SELECT
    s.label          AS season,
    g.game_date,
    ht.name          AS home_team,
    g.home_score,
    g.away_score,
    at.name          AS away_team,
    g.overtime
FROM games g
JOIN seasons s  ON s.season_id = g.season_id
JOIN teams ht  ON ht.team_id = g.home_team_id
JOIN teams at  ON at.team_id = g.away_team_id
WHERE s.label = '2023-24'
    AND g.status = 'Finished'
ORDER BY g.game_date
LIMIT 10;
```

Листинг В.3 — Агрегация сезонной статистики по позициям

```
SELECT
    p.code                AS position,
    COUNT(DISTINCT pss.player_id) AS player_count,
    ROUND(AVG(pss.avg_pts), 2)    AS avg_pts,
    ROUND(AVG(pss.avg_reb), 2)    AS avg_reb,
    ROUND(AVG(pss.avg_ast), 2)    AS avg_ast,
    ROUND(AVG(pss.per), 2)        AS avg_per
FROM player_season_stats pss
```

```

JOIN players pl ON pl.player_id = pss.player_id
JOIN positions p ON p.position_id = pl.position_id
JOIN seasons s ON s.season_id = pss.season_id
WHERE s.label = '2023-24'
      AND pss.avg_min >= 15
GROUP BY p.code
ORDER BY avg_per DESC;

```

Листинг В.4 — Сравнение PER игрока со средним по лиге

```

SELECT
    pl.first_name || ' ' || pl.last_name AS player,
    t.abbreviation AS team,
    pss.per,
    pss.avg_pts
FROM player_season_stats pss
JOIN players pl ON pl.player_id = pss.player_id
JOIN teams t ON t.team_id = pss.team_id
WHERE pss.season_id = (SELECT season_id FROM seasons WHERE label =
    '2023-24')
      AND pss.per > (
        SELECT AVG(per)
        FROM player_season_stats
        WHERE season_id = (SELECT season_id FROM seasons WHERE label
            = '2023-24')
            AND per IS NOT NULL
        )
ORDER BY pss.per DESC
LIMIT 10;

```

Листинг В.5 — Объединение топ-5 по очкам и передачам

```

(SELECT 'Scoring' AS category,
    pl.first_name || ' ' || pl.last_name AS player,
    pss.avg_pts AS value
FROM player_season_stats pss
JOIN players pl ON pl.player_id = pss.player_id
WHERE pss.season_id = (SELECT season_id FROM seasons WHERE label =
    '2023-24'))
ORDER BY pss.avg_pts DESC
LIMIT 5)

UNION ALL

```

```

(SELECT 'Assists',
      pl.first_name || ' ' || pl.last_name,
      pss.avg_ast
FROM player_season_stats pss
JOIN players pl ON pl.player_id = pss.player_id
WHERE pss.season_id = (SELECT season_id FROM seasons WHERE label =
      '2023-24'))
ORDER BY pss.avg_ast DESC
LIMIT 5);

```

Приложение Г. Скрипты создания функций и процедур

```

CREATE OR REPLACE FUNCTION calculate_efg(
  p_player_id INT,
  p_season_id INT,
  p_team_id    INT DEFAULT NULL
)
RETURNS DECIMAL(5,4)
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
  v_fgm NUMERIC := 0;
  v_fg3m NUMERIC := 0;
  v_fga NUMERIC := 0;
BEGIN
  SELECT
    COALESCE(SUM(gps.fgm), 0),
    COALESCE(SUM(gps.fg3m), 0),
    COALESCE(SUM(gps.fga), 0)
  INTO v_fgm, v_fg3m, v_fga
  FROM game_player_stats gps
  JOIN games g ON g.game_id = gps.game_id
  WHERE gps.player_id = p_player_id
    AND g.season_id = p_season_id
    AND (p_team_id IS NULL OR gps.team_id = p_team_id);

  IF v_fga = 0 THEN
    RETURN NULL;
  END IF;

```



```

        RETURN ROUND((v_fgm + 0.5 * v_fg3m) / v_fga, 4);
END;
$$;

```

Листинг Г.1 — Реализация функции calculate_ts (фрагмент)

```

CREATE OR REPLACE FUNCTION calculate_ts(
    p_player_id INT, p_season_id INT, p_team_id INT DEFAULT NULL
) RETURNS DECIMAL(5,4) LANGUAGE plpgsql STABLE AS $$
DECLARE
    v_pts NUMERIC; v_fga NUMERIC; v_fta NUMERIC; v_denom NUMERIC;
BEGIN
    SELECT COALESCE(SUM(gps.points), 0),
           COALESCE(SUM(gps.fga), 0),
           COALESCE(SUM(gps.fta), 0)
    INTO v_pts, v_fga, v_fta
    FROM game_player_stats gps
    JOIN games g ON g.game_id = gps.game_id
    WHERE gps.player_id = p_player_id
           AND g.season_id = p_season_id
           AND (p_team_id IS NULL OR gps.team_id = p_team_id);

    IF v_fga = 0 AND v_fta = 0 THEN RETURN NULL; END IF;

    v_denom := 2.0 * (v_fga + 0.44 * v_fta);
    IF v_denom = 0 THEN RETURN NULL; END IF;

    RETURN ROUND(v_pts::DECIMAL / v_denom, 4);
END; $$;

```

Листинг Г.2 — Триггер проверки корректности очков (фрагмент)

```

CREATE OR REPLACE FUNCTION trg_validate_team_score_row()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_sum INT; v_expected INT;
BEGIN
    IF v_expected IS NOT NULL AND v_sum <> v_expected THEN
        RAISE EXCEPTION
            'Player points sum (%) does not match team score (%)',
            v_sum, v_expected;
    END IF;

```

```
        RETURN COALESCE(NEW, OLD);  
    END; $$;
```

Листинг Г.3 — Разграничение доступа командами привилегий

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;  
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO analyst;  
  
GRANT SELECT ON v_team_stats TO reader;  
GRANT SELECT ON v_top_players TO reader;
```

Приложение Д. Исходный код механизма кэширования

В приложении приведён обобщённый декоратор с ключом по пути и строке запроса. В реализованных маршрутах API (листинг 3.4, технологический раздел) применяется явный формат ключа `leaders:{metric}:{season_id}:...`, что обеспечивает точечную инвалидацию по шаблонам при вызове `POST /admin/refresh`.

```
import json  
from functools import wraps  
from fastapi import Request, Response  
  
def cache_response(ttl_seconds: int = 300):  
    def decorator(func):  
        @wraps(func)  
        async def wrapper(*args, **kwargs):  
            request: Request = kwargs.get("request")  
            redis_client = request.app.state.redis  
  
            cache_key = f"api_cache:{request.url.path}?{request.url  
.query}"  
  
            cached_data = await redis_client.get(cache_key)  
            if cached_data:  
                return Response(  
                    content=cached_data,  
                    media_type="application/json"  
                )  
  
            response = await func(*args, **kwargs)  
  
            await redis_client.setex(
```

```

        cache_key,
        ttl_seconds,
        json.dumps(response)
    )
    return response
    return wrapper
    return decorator

```

Приложение Е. Пользовательский интерфейс программного комплекса

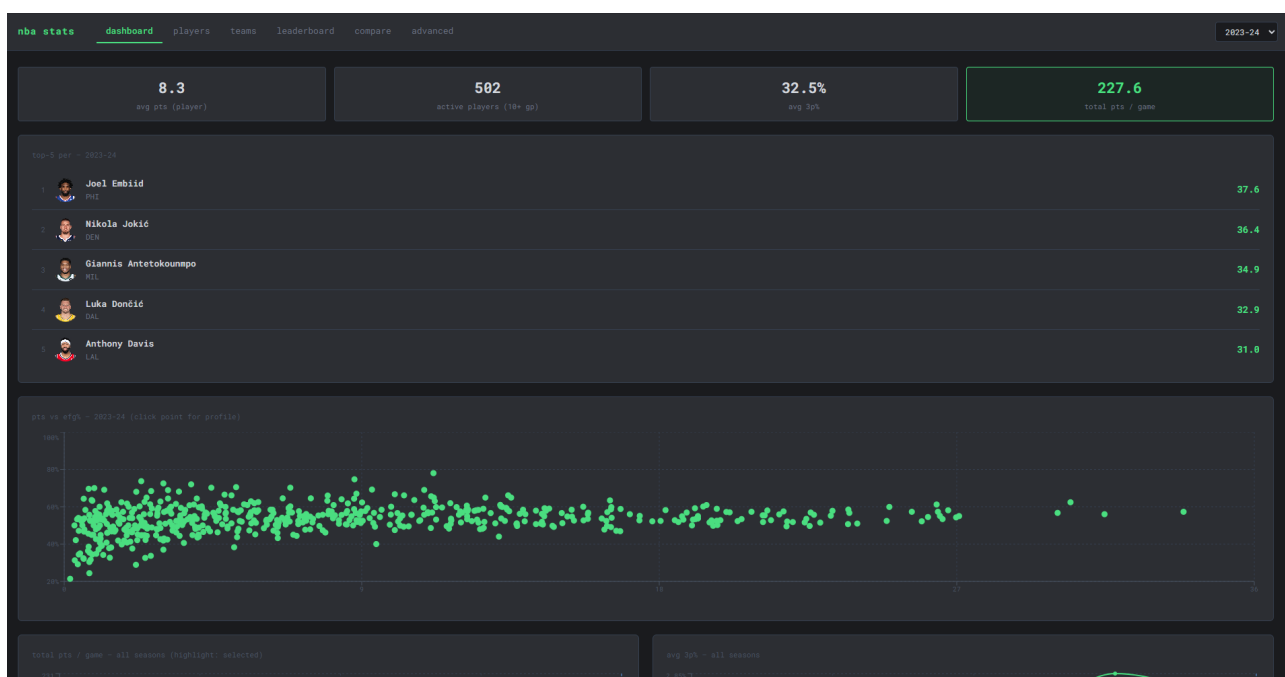


Рисунок Е.1 — Главная страница приложения (Dashboard)

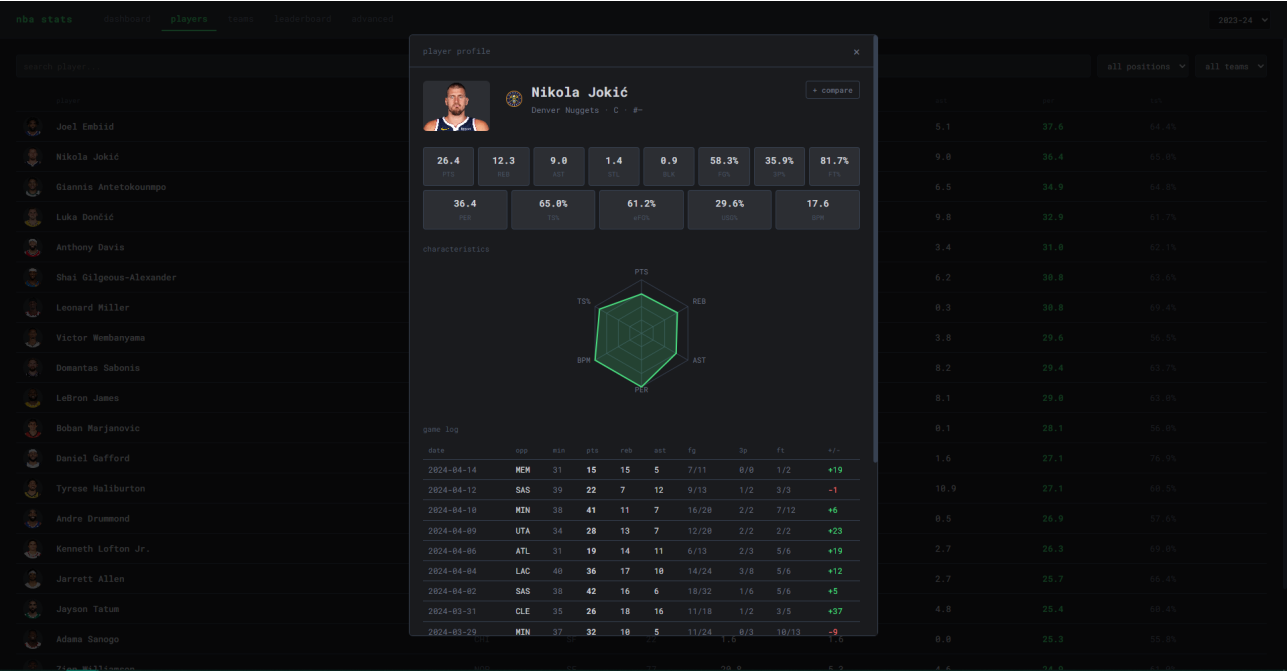


Рисунок Е.2 — Модальное окно профиля игрока: сезонные метрики и игровой журнал

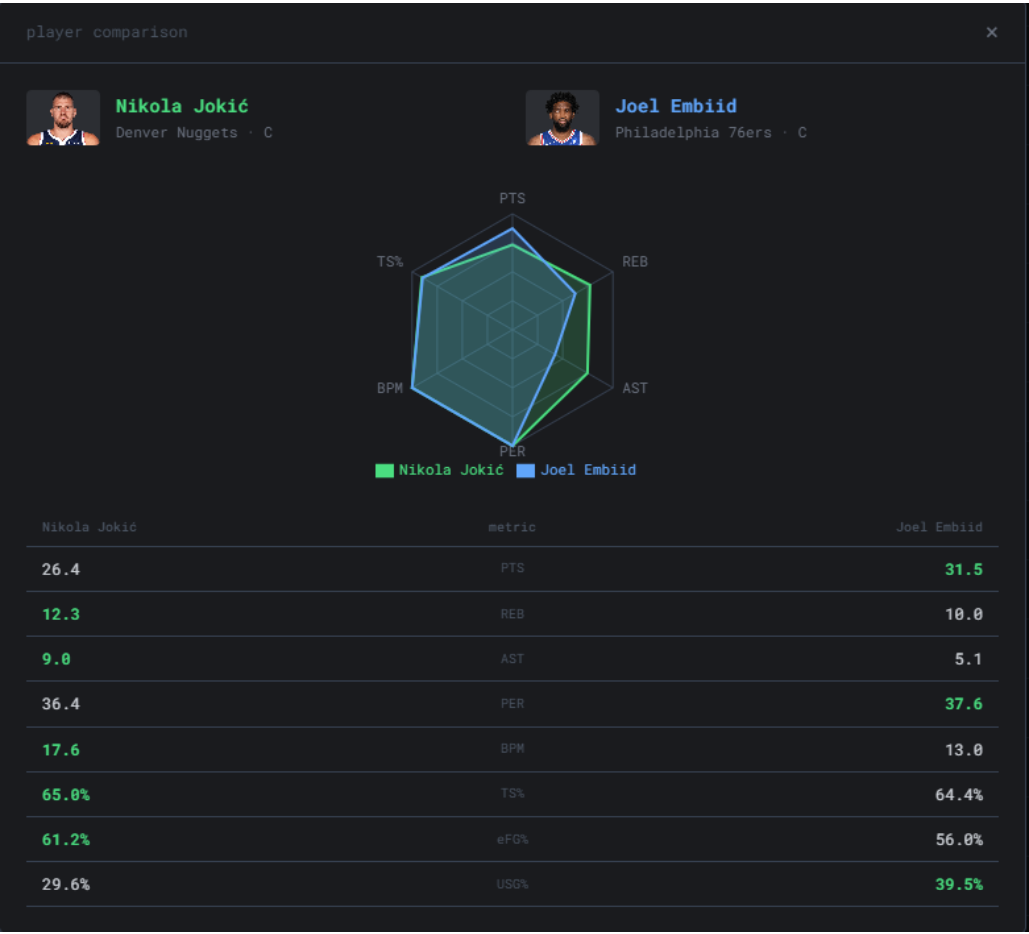


Рисунок Е.3 — Модальное окно сравнения профилей двух игроков

Приложение Ж. Презентация

презентация